

Embedded System Software Engineer Technical Interview Guide

By Don Hong

draft-2 · 2026-09

목차

By Don Hong · draft-2 · 2026-09

draft-1(31p)을 기반으로 오류를 바로잡고, **const**와 **포인터** 장을 새로 쓰고, 메모리·인터럽트·통신 프로토콜 장과 **기출 문제 5개**(부록 A)를 추가한 개정판. 부록의 모범 답안 코드는 모두 `cc -std=c11 -Wall -Wextra -Wconversion` + AddressSanitizer/UBSan으로 컴파일·테스트를 통과했다.

draft-2 변경 요약

서문: 기술 면접의 본질 - "왜"를 묻는 이유

I. C 언어 핵심 한정자: volatile, static, const

- A. volatile: 컴파일러 최적화의 제어
- B. static: 범위(Scope)와 생명주기(Lifetime)의 통제
- C. const: "읽기 전용"이라는 계약

II. 포인터와 const 완전 정복

- A. 포인터 기초 다시 보기
- B. const와 포인터의 4가지 조합
- C. 이중 포인터와 const: 왜 `char **` → `const char **`는 안 되는가
- D. volatile + const + 포인터: 레지스터 선언의 정석
- E. typedef와 문자열 리터럴의 함정
- F. 함수 포인터: 콜백과 디스패치 테이블
- G. 고급 포인터 주제
- H. 면접 퀴즈: 포인터와 const

III. 하드웨어 직접 제어: 메모리 매핑과 비트 조작

- A. 메모리 맵(Memory Map)과 Bare Metal 레지스터 접근
- B. 비트 조작(Bit Manipulation) 테크닉

IV. 메모리 구조와 정수 함정

- A. 펌웨어 메모리 레이아웃
- B. 구조체 패딩과 정렬
- C. 정수 승격과 부호 변환의 함정

V. 인터럽트와 동시성

- A. ISR 작성 원칙
- B. 공유 데이터 보호: 크리티컬 섹션
- C. 락 없는 SPSC 원형 버퍼 (ISR → main)
- D. 재진입성(Reentrancy)과 스레드 안전성
- E. 워치독(Watchdog)

VI. RTOS 핵심 동기화 기법: 뮤텁스와 세마포어

- A. 뮤텁스(Mutex) vs. 세마포어(Semaphore): 핵심 차이
- B. 우선순위 역전(Priority Inversion) 문제와 해결

VII. 임베디드 시스템을 위한 핵심 자료구조 구현

- A. 단일 연결 리스트(Singly Linked List)

B. 원형 버퍼(Circular Buffer / Ring Buffer)

VIII. C/C++ 코드 분석: 숨겨진 동작과 고급 패턴

A. C 코드: printf와 정의되지 않은 동작 (Undefined Behavior)

B. C++ 코드: 객체 생명주기와 리소스 관리

IX. 통신 프로토콜 기초: UART, I2C, SPI

A. 비교표

B. 자주 나오는 심화 질문

X. 알고리즘 문제 해결: LeetCode 패턴과 C 구현 관점

A. LeetCode (Easy/Medium) 핵심 패턴

B. 'Pythonic'한 코드 활용법

C. 면접 진행 방식 (언어와 무관)

XI. 결론: "왜"를 이해하는 엔지니어

부록 A. 기출 문제 풀이

A-1. 32비트 타이머 2개로 64비트 시간 읽기

A-2. atoi 구현

A-3. Reverse Bits

A-4. Divide Two Integers

A-5. Palindrome (문자열과 정수)

부록 B. draft-1 정오표

draft-2 변경 요약

- **신규 장:** II. 포인터와 const 완전 정복 · IV. 메모리 구조와 정수 함정 · V. 인터럽트와 동시성 · IX. 통신 프로토콜 기초
- **신규 절:** I-C. const 기초, III-B의 필드(여러 비트) 매크로·W1C 레지스터·엔디안 판별, VI의 FreeRTOS ISR API·우선순위 상한, VII의 이중 포인터 삭제·SPSC 링버퍼, VIII의 가상 소멸자·임베디드 C++
- **부록 A. 기출 문제 5개:** 32비트 타이머 2개로 64비트 시간 읽기, atoi, reverse bits, divide two integers, palindrome. 각 문제마다 원본 코드 리뷰 + 검증된 모범 답안 + 꼬리 질문
- **부록 B. draft-1 정오표:** 기술적으로 틀렸거나 오해 소지가 있던 문장 10곳 수정 내역

서문: 기술 면접의 본질 - "왜"를 묻는 이유

본 문서는 임베디드 시스템, 펌웨어, 또는 시스템 소프트웨어 엔지니어로서의 이직을 준비하는 동료 엔지니어들을 위한 심층 기술 참고 노트입니다. 이 문서는 단순한 질의응답 목록이 아니며, 면접관이 특정 질문을 하는 근본적인 이유와 그 이면에 숨겨진 핵심 원리를 탐구하는 것을 목표로 합니다.

임베디드 시스템 면접관은 지원자가 volatile의 정의를 단순히 암기하는 것보다, "컴파일러 최적화"라는 개념과 그것이 하드웨어와 상호작용할 때 발생하는 근본적인 문제를 이해하고 있는지 확인하고자 합니다. 마찬가지로, static 키워드에 대해 질문하는 것은 C언어의 "연결성(linkage)"과 "정보 은닉(information hiding)" 원칙을 이해하는지, const와 포인터의 조합을 묻는 것은 "누가 무엇을 바꿀 수 있는가"라는 인터페이스 계약을 코드로 표현할 줄 아는지, 뮤텍스와 세마포어의 차이를 묻는 것은 "우선순위 역전(priority inversion)"이라는 치명적인 실시간 시스템의 함정을 인지하고 있는지 평가하기 위함입니다.

이 참고 노트는 각 주제에 대한 기술적 "정답"과 함께, 그 이면에 있는 "핵심 원리"와 "면접관의 의도"를 함께 제공합니다. 이를 통해 독자가 어떠한 변형된 질문에도 흔들림 없이 대응하고, 자신의 깊은 공학적 이해를 증명할 수 있도록 돕는 것이 본 문서의 목적입니다.

I. C 언어 핵심 한정자: volatile, static, const

C언어에서 volatile, static, const는 하드웨어에 밀접하게 접근하고 시스템의 상태를 관리해야 하는 임베디드 엔지니어에게 가장 중요한 키워드입니다. 면접관은 이 키워드들을 통해 지원자가 컴파일러의 동작 방식, 메모리 모델, 링커의 역할을 얼마나 깊이 이해하고 있는지 측정합니다.

A. volatile: 컴파일러 최적화의 제어

volatile 키워드는 C/C++ 표준의 일부로, 변수의 값이 "프로그램의 현재 실행 흐름과 무관하게" 언제든지 변경될 수 있음을 컴파일러에게 알리는 한정자(qualifier)입니다. C 표준은 volatile 객체에 대한 접근을 **관찰 가능한 동작(observable behavior)**으로 취급합니다. 즉 "이 변수에 대한 읽기와 쓰기는 소스에 적힌 횟수와 순서 그대로 수행하라"는 의미입니다.

1. 핵심 메커니즘: 컴파일러 최적화 억제

문제 상황 (최적화의 함정):

최신 컴파일러는 코드 실행 속도를 높이기 위해 매우 공격적인 최적화를 수행합니다. 변수의 값을 메모리에서 한 번 읽어온 후, 그 값을 CPU 레지스터에 캐싱(caching)합니다. 이후 해당 변수를 다시 참조할 때, 컴파일러가 판단하기에 그사이 값이 변경된 적이 없다면(즉, 현재 코드 블록 내에 해당 변수에 값을 할당하는 코드가 없다면), 메모리 접근을 다시 수행하지 않고 CPU 레지스터에 저장된 값을 재사용합니다.

임베디드에서의 실패 시나리오:

하드웨어 상태 레지스터(Status Register)의 특정 비트가 1이 될 때까지 기다리는 코드를 가정해 봅시다.

```
/* 'HW_STATUS_REGISTER'는 0x40001000 주소에 매핑된 하드웨어 레지스터 */
uint32_t *p_status = (uint32_t *)0x40001000;

/* 데이터 수신 완료(bit 0)를 기다림 */
while ((*p_status & 0x01) == 0) {
    /* wait */
}
```

최적화 컴파일러는 이 코드를 다음과 같이 분석합니다.

1. `*p_status`의 값을 메모리에서 읽어와 레지스터 R0에 저장한다.
2. while 루프 본문 안에는 `*p_status`의 값을 변경하는 코드가 존재하지 않는다.
3. 따라서 `*p_status`의 값은 루프 안에서 변하지 않는다.
4. 처음 읽은 값이 0이었다면, 이 루프는 `while (1);`과 동일한 무한 루프이다.

실제 하드웨어가 데이터 수신을 완료하고 0번 비트를 1로 바꾸더라도, 프로그램은 캐시된 0의 값만 영원히 바라보게 됩니다. 이런 버그는 `-O0` 디버그 빌드에서는 재현되지 않고 `-O2` 릴리스 빌드에서만 나타나기 때문에 특히 찾기 어렵습니다.

volatile의 해결책:

```

/* 'volatile' 대상으로 선언: 가리키는 값이 volatile */
volatile uint32_t *p_status = (volatile uint32_t *)0x40001000;

while ((*p_status & 0x01) == 0) {
    /* wait */
}

```

이제 컴파일러는 루프가 반복될 때마다 0x40001000 주소에서 값을 매번 다시 읽는 코드를 생성합니다. `volatile`의 위치가 `*`의 왼쪽이라는 점에 주목하세요. 바뀌는 것은 포인터가 아니라 **포인터가 가리키는 레지스터 값**입니다(II장 참조).

2. 면접 함정: volatile이 보장하지 않는 것

`volatile`의 의미는 "접근을 생각하거나 합치지 말라"는 것, 그 이상도 이하도 아닙니다. 면접관은 지원자가 `volatile`을 멀티스레딩이나 원자성을 위한 만병통치약으로 오해하는지 확인하려 합니다.

- **volatile은 원자성(atomicity)을 보장하지 않는다.** CPU 워드보다 큰 변수는 여러 명령으로 나뉘어 읽힙니다. AVR 같은 8비트 MCU에서 `volatile uint32_t`는 바이트 단위 명령 4개로, 32비트 Cortex-M에서도 `volatile uint64_t`는 `LDR` 두 번으로 읽힙니다. 그 사이에 ISR이 값을 바꾸면 **찢어진 값(torn value)**, 즉 갱신 전 절반과 갱신 후 절반이 섞인 값을 읽게 됩니다. `count++` 같은 read-modify-write 역시 `volatile`이어도 원자적이지 않습니다. 해결책은 인터럽트 잠시 끄기(critical section, V장), RTOS 뮤텝스, C11 `<stdatomic.h>` (툴체인 지원 시) 또는 C++ `std::atomic`입니다.
- **volatile은 non-volatile 메모리 쓰기의 순서를 보장하지 않는다.** 아래 코드에서 (2)는 `volatile` 접근이라 반드시 수행되지만, (1)은 일반 메모리 쓰기이므로 관찰 가능한 동작이 아닙니다. 컴파일러는 (1)을 뒤로 미루거나, 이후에 같은 곳을 다시 쓰면 아예 생략(dead store elimination)할 수 있습니다.

```

int *data_ptr = ...;          /* 일반 메모리 */
volatile int *flag_ptr = ...; /* HW 레지스터 */

*data_ptr = 0xDEADBEEF; /* (1) 데이터 쓰기: 순서/존재가 보장되지 않음 */
*flag_ptr = 1;          /* (2) 플래그 설정: 반드시, 이 순서로 수행 */

```

하드웨어(예: DMA 컨트롤러)가 (1)의 메모리를 (2) 직후에 읽는다면, (1)의 메모리도 `volatile`로 접근해야 순서가 보장됩니다. 참고로 멀티코어 커널에서 말하는 "memory barrier"(Linux `smp_mb()` 등)는 CPU 간 메모리 가시성 문제를 다루는 별개의 개념이며 `volatile`과 관계가 없습니다.

3. 임베디드 시스템에서의 필수 사용 사례

1. **메모리 매핑 I/O (MMIO):** 하드웨어 페리페럴 레지스터에 접근할 때 (III-A 참조).
2. **ISR과 공유하는 변수:** ISR이 `g_flag = 1;`을 설정하고 메인 루프가 `while (g_flag == 0);`로 폴링하는 경우. 컴파일러는 ISR이 언제 호출되는지 모르므로, `volatile`이 없으면 루프 안에서 `g_flag`를 다시 읽지 않을 수 있습니다.
3. **멀티태스킹 환경의 공유 변수:** 단, 원자성이 보장되지 않으므로 이 용도로는 `volatile`만으로는 부족하고 뮤텝스나 `atomic` 연산이 함께 필요합니다.
4. **setjmp/longjmp:** `setjmp` 이후 수정되고 `longjmp` 후에 읽히는 지역 변수는 `volatile`이어야 값이 보장됩니다.

B. static: 범위(Scope)와 생명주기(Lifetime)의 통제

static은 C/C++에서 가장 혼란스러운 키워드 중 하나입니다. 그 의미는 사용되는 컨텍스트에 따라 완전히 달라집니다. static은 변수의 **저장 기간(Storage Duration)** 또는 식별자의 **연결(Linkage)** 속성을 제어합니다.

면접관은 "static이 무엇인가요?"라는 질문을 통해 지원자가 이 컨텍스트를 구분하는지 확인하려 합니다. "어떤 컨텍스트의 static을 말씀하시는 건가요?"라고 되묻거나, 아래 4가지 경우를 모두 설명할 수 있어야 합니다.

1. static 키워드의 4가지 용도 비교

표 1: static 키워드의 4가지 용도 비교

컨텍스트	핵심 속성 제어	생명주기	범위 (Scope)	연결성 (Linkage)	핵심 목적
함수 내 지역 변수	저장 기간	Static (프로그램 전체)	함수 블록 내	None	호출 간 값 유지
파일 범위 전역 변수	연결	Static (프로그램 전체)	파일(TU) 전체	Internal	정보 은닉 (모듈 내부 상태)
파일 범위 함수	연결	N/A (코드)	파일(TU) 전체	Internal	정보 은닉 (내부 함수)
C++ 클래스 멤버	클래스 귀속	Static (프로그램 전체)	클래스 (접근 지정자 따름)	External (기본값)	인스턴스 간 데이터 공유

2. 컨텍스트별 상세 분석

• a) 함수 내 지역 변수 (Local Static Variable)

- 생명주기: 일반 지역 변수(auto)는 호출 시 스택에 생기고 반환 시 사라집니다. static 지역 변수는 스택이 아니라 `.data` (초기값 있음) 또는 `.bss` (0 초기화) 영역에 있으며 프로그램 전체 기간 동안 존재합니다.
- 초기화 시점: **C에서는** 초기값이 상수 표현식이어야 하며 main 실행 전(startup 코드가 .data 복사, .bss 0 채움을 할 때) 이미 초기화되어 있습니다. **C++에서는** 비상수 초기화도 허용되며, 선언을 처음 지나갈 때 한 번 초기화됩니다(C++11부터 스레드 안전 보장, 이를 위한 guard 코드가 추가되므로 bare-metal에서는 `-fno-threadsafe-statics` 를 쓰기도 함).
- 사용 예: `int get_call_count(void) { static int counter = 0; return ++counter; }` 는 호출할 때마다 1, 2, 3...을 반환합니다.
- **함정**: static 지역 변수를 쓰는 함수는 **재진입 불가(non-reentrant)**입니다. ISR과 메인 루프가 동시에 호출하거나 두 태스크가 호출하면 상태가 섞입니다. 표준 라이브러리의 `strtok` 이 대표적인 예이며, 그래서 `strtok_r` 이 존재합니다.

• b) 파일 범위 전역 변수/함수 (File-Scope Static)

- C언어의 캡슐화 및 **정보 은닉**을 위한 핵심 도구입니다.
- 파일 범위에서 선언된 변수나 함수는 기본적으로 **외부 연결(External Linkage)**을 가지므로 다른 .c 파일에서 `extern` 으로 접근할 수 있습니다. static을 붙이면 **내부 연결(Internal Linkage)**로 바뀌어 해당 Translation Unit 밖에서는 보이지 않습니다.
- 실무 적용: 드라이버 모듈(예: uart.c)의 내부 버퍼(`static uint8_t uart_rx_buffer[64];`)나 내부 헬퍼(`static void uart_handle_rx(void)`)는 static으로 선언합니다. 외부 모듈의 실수 접근과 이름 충돌을 원천 차단하고, `uart_init()`, `uart_send()` 같은 공개 API만 헤더에 선언해 external로 남깁니다. 부가 효과로 컴파일러가 해당 함수를 인라인하거나 미사용 시 제거하기 쉬워집니다.
- **함정**: 헤더 파일에 `static int g_count;` 를 선언하면, 그 헤더를 include한 모든 .c 파일이 **각자 별개의 복사본**을 갖습니다. 공유 전역이 필요하면 헤더에는 `extern int g_count;` , 정의는 .c 한 곳에 둡니다. 반면 헤더의 `static`

`inline` 함수는 의도된 일반적 패턴입니다.

• c) C++ 클래스 멤버 (Class Static Members)

- static 멤버는 특정 인스턴스가 아니라 클래스 자체에 귀속됩니다.
- static data member: 모든 인스턴스가 공유하는 단 하나의 변수입니다 (예: `Car::object_count` 로 생성된 객체 수 세기).
- static member function: 인스턴스 없이 `MyClass::my_static_func()` 형태로 호출하며, this 포인터가 없으므로 static 멤버에만 접근할 수 있습니다. this가 없다는 성질 때문에 C 스타일 콜백(예: ISR 벡터, RTOS 태스크 진입점)으로 등록할 수 있는 멤버 함수는 static 멤버 함수뿐입니다.

C. const: "읽기 전용"이라는 계약

const는 "상수(constant)"가 아니라 "이 이름(경로)을 통해서는 수정하지 않겠다"는 읽기 전용 계약입니다. 이 구분이 const 관련 면접 질문의 대부분을 풀어줍니다. (포인터와의 조합은 II장에서 자세히 다룹니다.)

1. const가 하는 일 세 가지

- **컴파일러 검사:** const 객체에 대입하면 컴파일 에러가 납니다. 실수로 설정 테이블이나 입력 버퍼를 덮어쓰는 버그를 컴파일 타임에 잡습니다.
- **인터페이스 문서화:** `size_t strlen(const char *s)`, `void *memcpy(void *dst, const void *src, size_t n)` 처럼 함수 시그니처만 보고도 "이 함수는 입력을 바꾸지 않는다"는 것을 알 수 있습니다. 호출자는 문자열 리터럴이나 읽기 전용 데이터를 안심하고 넘깁니다.
- **메모리 배치:** 정적 저장 기간을 갖는 const 객체(전역 또는 `static const`)는 보통 `.rodata` 섹션에 놓이고, MCU에서는 플래시에 남아 RAM을 소비하지 않습니다. CRC 테이블, sine 테이블, 폰트 같은 큰 룩업 테이블에 `static const` 를 붙이는 이유입니다. 반면 함수 안의 non-static `const` 지역 변수는 여전히 스택에 생깁니다.

2. C의 const vs C++의 const vs #define vs enum

항목	C <code>const int N = 10;</code>	C++ <code>const int N = 10;</code>	<code>#define N 10</code>	<code>enum { N = 10 };</code>
타입 검사	O	O	X (텍스트 치환)	O (int)
파일 범위 배열 크기로 사용	X (상수 표현식 아님)	O	O	O
<code>switch</code> 의 <code>case</code> 라벨	X	O	O	O
파일 범위 기본 연결성	External	Internal	해당 없음	해당 없음
디버거에서 이름으로 보임	O	O	X	O
메모리 차지	가능(주소를 쓰면)	보통 없음(상수 접합)	없음	없음

C에서 "진짜 상수"가 필요하면 `enum` 또는 `#define`, C++에서는 `constexpr` 을 씁니다. C23부터는 C에도 `constexpr` 이 도입되었지만 임베디드 툴체인 지원은 아직 제한적입니다.

3. const를 벗겨내는 캐스트

```
const int limit = 100;
int *p = (int *)&limit; /* 캐스트로 const 제거: 컴파일은 된다 */
*p = 200;                /* UB: 원래 const로 정의된 객체 수정 */
```

원래 const로 **정의된** 객체를 수정하는 것은 정의되지 않은 동작입니다. 플래시에 배치된 경우 쓰기가 무시되거나 HardFault가 나고, 컴파일러가 `limit` 을 100으로 상수 접힘(constant folding)했다면 코드 곳곳에서 여전히 100을 사용합니다. 반면 원래 non-const인 객체를 const 포인터로 받았다가 캐스트해서 수정하는 것은 합법입니다(`strchr` 이 `const char *` 를 받아 `char *` 를 반환하는 이유).

4. const와 volatile은 함께 쓸 수 있다

"변수가 const이면서 volatile일 수 있나요?"는 단골 질문입니다. 답은 **예**입니다. 읽기 전용 하드웨어 상태 레지스터가 대표 예로, **내 코드는 쓰면 안 되고(const), 하드웨어는 값을 바꾼다(volatile)**는 뜻입니다. ARM CMSIS 헤더는 이를 `#define __I volatile const` (읽기 전용), `#define __O volatile` (쓰기 전용), `#define __IO volatile` (읽기/쓰기)로 정의해 레지스터 구조체에 사용합니다.

II. 포인터와 const 완전 정복

포인터는 C의 가장 강력한 도구이자 가장 흔한 버그의 원천입니다. 임베디드 면접에서 포인터 질문은 "문법을 외웠는가"보다 "메모리를 머릿속에 그릴 수 있는가", 그리고 "const로 소유권과 수정 권한을 표현할 줄 아는가"를 봅니다.

A. 포인터 기초 다시 보기

1. 포인터 = 주소 + 타입

포인터 값은 주소이지만, 포인터 **타입**이 두 가지를 결정합니다.

- **역참조 시 몇 바이트를 어떻게 해석할지:** `uint8_t *`는 1바이트, `uint32_t *`는 4바이트를 읽습니다.
- **포인터 연산의 보폭:** `p + 1`은 주소에 1이 아니라 `sizeof(*p)`를 더합니다. `uint32_t *p = (uint32_t *)0x40000000;`이면 `p + 1`은 `0x40000004`입니다. 레지스터 오프셋 계산에서 자주 틀리는 부분입니다.

`void *`는 "타입 없는 주소"로, 역참조할 수 없고 표준 C에서는 포인터 연산도 할 수 없습니다(GCC는 확장으로 1바이트 보폭 허용). 바이트 단위로 다루려면 `uint8_t *`로 변환합니다.

2. 배열과 포인터는 다르다

```
int a[5];
int *p = a;          /* 배열 이름은 대부분의 식에서 첫 원소 주소로 변환(decay) */

sizeof(a);           /* 20 : 배열 전체 크기 (int 4바이트 가정) */
sizeof(p);           /* 4 또는 8 : 포인터 크기 */
&a;                 /* 타입 int (*)[5] : "배열 전체"를 가리키는 포인터 */
&a + 1;             /* 배열 전체(20바이트) 다음 주소 */
&a[0] + 1;          /* 다음 원소(4바이트 뒤) 주소 */

void f(int arr[]);   /* 함수 매개변수의 배열 표기는 실제로 int *arr 과 같다 */
                    /* f 안에서 sizeof(arr)는 포인터 크기 → 길이를 따로 넘겨야 함 */
```

면접 포인트: "함수에 배열을 넘기면 왜 길이를 같이 넘기나요?"라는 질문의 답이 바로 decay입니다. `#define ARRAY_SIZE(x) (sizeof(x) / sizeof((x)[0]))` 매크로는 **배열에만** 동작하고 포인터에 쓰면 조용히 틀린 값을 냅니다.

3. 위험한 포인터 세 가지

- **NULL 포인터:** 역참조하면 UB. 많은 Cortex-M에서 주소 0은 플래시(벡터 테이블)라서 **크래시 없이 쓰레기 값을 읽는** 경우가 있어 데스크톱보다 더 위험합니다. MPU로 0번지 접근을 막는 방법도 있습니다.
- **댕글링(dangling) 포인터:** 해제된 메모리나 반환된 스택 프레임 가리키는 포인터. `return &local_var;`가 전형적인 예입니다.
- **와일드(wild) 포인터:** 초기화되지 않은 포인터. 지역 포인터는 선언과 동시에 초기화하는 습관이 필요합니다.

B. const와 포인터의 4가지 조합

포인터 선언에는 const를 둘 수 있는 위치가 두 곳입니다. `*`의 왼쪽에 있으면 가리키는 데이터가, `*`의 오른쪽에 있으면 포인터 자체가 읽기 전용입니다.

선언	읽는 법 (오른쪽→왼쪽)	<code>*p = x</code> (값 변경)	<code>p = &y</code> (포인터 변경)	대표 용도
<code>const int *p</code>	p는 포인터, const int를 가리킴	불가	가능	함수 입력 매개변수 (<code>const char *str</code>)
<code>int const *p</code>	위와 완전히 동일	불가	가능	위와 동일 (스타일 차이)
<code>int *const p</code>	p는 const 포인터, int를 가리킴	가능	불가	고정 주소 레지스터, 고정 버퍼
<code>const int *const p</code>	p는 const 포인터, const int를 가리킴	불가	불가	읽기 전용 고정 테이블/레지스터

읽는 규칙 (오른쪽-왼쪽 규칙): 변수 이름에서 출발해 오른쪽→왼쪽으로 읽고, `*`는 "pointer to"로 읽습니다. `const`는 바로 왼쪽의 것에 걸리며, 맨 왼쪽에 있을 때만 바로 오른쪽의 타입에 걸립니다.

- `int *const p` → "p is a **const pointer** to int"
- `const int *p` → "p is a pointer to **int that is const**"
- `char *const *pp` → "pp is a pointer to a **const pointer** to char"

아래 결과는 clang 21로 직접 확인한 진단 메시지입니다.

```
int x = 1, y = 2;

const int *p1 = &x;
*p1 = 3;          /* error: read-only variable is not assignable */
p1 = &y;          /* OK */

int *const p2 = &x;
*p2 = 3;          /* OK */
p2 = &y;          /* error: cannot assign to variable 'p2' with const-qualified type 'int *const' */

const int *const p3 = &x;
p3 = &y;          /* error */
```

1. 암묵 변환 규칙: const는 더하기는 쉽고 빼기는 어렵다

```
int x = 1;
int *ip = &x;
const int *cip = ip; /* OK: T* → const T* 는 안전한 방향이라 암묵 변환 */
int *ip2 = cip;      /* C: warning (discards qualifiers), C++: error */
```

"쓸 수 있는 권한"을 "읽기만 하는 권한"으로 줄이는 것은 자동이지만, 반대 방향은 명시적 캐스트가 필요하고 C++에서는 에러입니다. 그래서 C 코드에서는 `-Werror=discarded-qualifiers` 같은 옵션으로 경고를 에러로 격상하는 팀이 많습니다.

2. 함수 매개변수에서의 const: 무엇이 의미 있나

```
void uart_send(const uint8_t *buf, size_t len); /* 의미 있음: 호출자 데이터를 안 바꾼다는 계약 */
void set_speed(const int rpm);                /* 선언에서는 의미 없음: 값 복사라 호출자와 무관 */
void fill(uint8_t *const buf, size_t len);     /* 선언에서는 의미 없음: 포인터 자체도 값 복사 */
```

가리키는 대상의 `const`(`*`의 왼쪽)는 호출자와의 계약이지만, 매개변수 자체의 `const`(`*`의 오른쪽 또는 값 매개변수)는 함수 내부 구현의 실수 방지일 뿐 인터페이스에는 영향이 없습니다. 선언에는 생략하고 정의에만 쓰는 스타일도 있습니다.

중요한 오해: `void f(const int *p)` 안에서 `*p`가 절대 바뀌지 않는 것은 아닙니다. `const`는 "p를 통해서 안 바꾼다"는 약속일 뿐이며, 같은 메모리를 가리키는 다른 non-const 포인터나 ISR, 하드웨어가 값을 바꿀 수 있습니다. 컴파일러가 `const`만 보고 `*p`를 캐싱할 수 없는 이유입니다(캐싱을 허용하려면 `restrict`가 필요, G절).

C. 이중 포인터와 const: 왜 `char **` → `const char **`는 안 되는가

직관적으로는 `char *` → `const char *`가 되니 `char **` → `const char **`도 될 것 같지만, 금지되어 있습니다(C는 경고, C++는 에러로 확인). 허용하면 `const` 객체를 수정할 수 있는 구멍이 생기기 때문입니다.

```
const char secret = 'A'; /* 진짜 읽기 전용 객체 (플래시에 있을 수 있음) */
char *p;
const char **pp = &p; /* 만약 이게 허용된다면... (실제로는 금지) */
*pp = &secret; /* OK로 보임: const char* 에 const char 주소 대입 */
*p = 'B'; /* p가 이제 secret을 가리킴 → const 객체 수정! */
```

안전한 형태는 중간 단계까지 `const`를 붙인 `const char *const *`입니다. C++은 이 변환을 암묵적으로 허용하고, C는 C23 이전까지 허용하지 않아 캐스트가 필요합니다. 실무에서는 `int main(int argc, char **argv)`를 `const char **`를 받는 함수에 넘길 때 이 경고를 만나게 됩니다.

D. volatile + const + 포인터: 레지스터 선언의 정석

II-B의 규칙은 `volatile`에도 똑같이 적용됩니다. 레지스터 선언에서 한정자의 위치가 정확히 무엇을 뜻하는지 설명할 수 있어야 합니다.

선언	의미	용도
<code>volatile uint32_t *p</code>	가리키는 값이 HW에 의해 바뀔. 포인터는 변경 가능	여러 레지스터를 순회하는 포인터
<code>volatile uint32_t *const REG</code>	값은 HW가 바꾸고 나도 읽고 씬. 주소는 고정	읽기/쓰기 레지스터
<code>const volatile uint32_t *const REG</code>	값은 HW가 바꾸지만 나는 읽기만. 주소 고정	읽기 전용 상태 레지스터 (쓰기 시도가 컴파일 에러)
<code>uint32_t *volatile p</code>	포인터 변수 자체가 비동기적으로 바뀔	ISR이 갱신하는 버퍼 포인터 (드뎌)

```

#define UART3_BASE 0x40013800UL

/* 방법 1: const 포인터 상수 */
static volatile uint32_t *const UART3_DR = (volatile uint32_t *) (UART3_BASE + 0x04);
static const volatile uint32_t *const UART3_SR = (const volatile uint32_t *) (UART3_BASE + 0x00);

*UART3_DR = 'A';          /* OK */
uint32_t sr = *UART3_SR;  /* OK */
*UART3_SR = 0;            /* 컴파일 에러: 읽기 전용 레지스터 보호 */

/* 방법 2: 역참조까지 포함한 매크로 (CMSIS/벤더 HAL 스타일) */
#define REG_UART3_DR (*(volatile uint32_t *) (UART3_BASE + 0x04))
REG_UART3_DR = 'A';

```

draft-10에서는 "포인터 변수를 쓰면 로드 명령이 하나 더 필요해 매크로가 더 빠르다"고 설명했습니다. 정확히는 **non-const 포인터 변수**일 때만 그렇습니다. 위의 `static ... *const` 포인터는 값이 컴파일 타임에 확정되므로 최적화 빌드에서 매크로와 **완전히 같은 명령**이 생성됩니다. 또한 ARM에서는 32비트 절대 주소를 명령어 하나에 담을 수 없어서 어느 방식이든 `LDR r1, =0x40013804` (리터럴 풀에서 주소 로드) 후 `STR r0, [r1]` 형태가 됩니다.

E. typedef와 문자열 리터럴의 함정

1. typedef는 텍스트 치환이 아니다

```

typedef char *pstr;
const pstr p;          /* const char *p 가 아니라 char *const p 이다! */

```

`pstr`은 이미 "char를 가리키는 포인터"라는 **하나의 타입**이고, `const`는 그 타입 전체(포인터)에 붙습니다. `#define pstr char *`였다면 `const char *p`가 되었을 것입니다. 이 차이를 묻는 문제는 typedef와 #define의 차이를 동시에 확인하는 좋은 질문이라 자주 나옵니다. 그래서 포인터를 typedef로 숨기지 말라는 코딩 규칙(Linux 커널 스타일 등)이 있습니다.

2. 문자열 리터럴은 수정하면 안 된다

```

char *s1 = "hello";      /* 리터럴은 .rodata(플래시). C에서는 타입이 char[]라 경고 없이 컴파일됨 */
s1[0] = 'H';            /* UB: 크래시 또는 무시 */

const char *s2 = "hello"; /* 올바른 선언 (C++에서는 const가 필수) */

char s3[] = "hello";     /* 리터럴을 초기값으로 복사한 6바이트 배열(RAM) */
s3[0] = 'H';            /* OK */

```

`sizeof(s1)`은 포인터 크기, `sizeof(s3)`은 6(널 문자 포함)입니다.

F. 함수 포인터: 콜백과 디스패치 테이블

펌웨어에서 함수 포인터는 인터럽트 벡터 테이블, 드라이버 콜백, 상태 머신, 명령어 디스패치 테이블의 기반입니다.

```
/* 선언 읽기: fp는 포인터, (int)를 받아 int를 반환하는 함수를 가리킴 */
int (*fp)(int);
int *f2(int);          /* 주의: 이것은 int*를 반환하는 "함수" 선언 */

/* typedef로 가독성 확보 */
typedef void (*cmd_handler_t)(const uint8_t *payload, size_t len);

static void cmd_ping(const uint8_t *p, size_t n);
static void cmd_reset(const uint8_t *p, size_t n);

/* const 테이블 → 플래시에 배치되고, 런타임에 덮어써서 코드 흐름을 탈취할 수 없음 */
static const cmd_handler_t cmd_table[] = {
    [0x01] = cmd_ping,
    [0x02] = cmd_reset,
};

void dispatch(uint8_t id, const uint8_t *payload, size_t len) {
    if (id < ARRAY_SIZE(cmd_table) && cmd_table[id] != NULL) {
        cmd_table[id](payload, len);    /* 범위 검사 없이 호출하면 임의 주소로 점프 */
    }
}
```

면접 포인트: 함수 포인터 테이블에 `const` 를 붙이는 이유(RAM 절약 + 보안), 인덱스 범위 검사의 중요성, 그리고 Cortex-M 벡터 테이블이 사실상 "주소 0(또는 VTOR)에 놓인 const 함수 포인터 배열"이라는 점을 연결해서 말할 수 있으면 좋습니다.

G. 고급 포인터 주제

1. 이중 포인터로 연결 리스트 노드 삭제 (head 특별 처리 없애기)

```
struct Node { int value; struct Node *next; };

/* pp는 "다음 노드를 가리키는 포인터 변수"의 주소: head 자체 또는 어떤 노드의 next 필드 */
void list_remove(struct Node **head, const struct Node *target) {
    struct Node **pp = head;
    while (*pp != NULL && *pp != target) {
        pp = &(*pp)->next;
    }
    if (*pp != NULL) {
        *pp = target->next; /* head든 중간이든 같은 코드로 처리 */
    }
}
```

prev 포인터와 "head를 지우는 경우" 분기가 사라집니다. `Node **` 가 무엇을 가리키는지 그림으로 설명할 수 있다면 포인터 이해도를 강하게 어필할 수 있습니다.

2. 바이트 버퍼를 구조체/정수로 캐스팅하지 말 것 (정렬 + strict aliasing)

```
uint8_t rx[8]; /* UART로 받은 패킷 */

uint32_t bad = *(uint32_t*)(rx + 1); /* 나쁜 예 */
uint32_t good;
memcpy(&good, rx + 1, sizeof good); /* 좋은 예 */
```

나쁜 예의 문제는 두 가지입니다. (1) **정렬(alignment)**: `rx + 1` 은 4의 배수 주소가 아닐 수 있고, 일부 아키텍처(Cortex-M0, 일부 DSP)에서는 정렬되지 않은 워드 접근이 HardFault를 냅니다. (2) **strict aliasing**: `uint8_t` 배열을 `uint32_t` lvalue로 읽는 것은 표준상 UB이며, `-O2` 의 `-fstrict-aliasing` 최적화가 코드를 예상과 다르게 만들 수 있습니다. `memcpy` 는 컴파일러가 단일 `LDR` 명령으로 최적화하므로 성능 손해도 없습니다. 패킷의 엔디안도 여전히 따로 처리해야 합니다.

3. restrict: "이 포인터만 이 메모리에 접근한다"는 약속

`void *memcpy(void *restrict dst, const void *restrict src, size_t n)` 의 `restrict`는 `dst`와 `src`가 겹치지 않는다고 컴파일러에 약속해 벡터화 같은 최적화를 허용합니다. 겹칠 수 있으면 `memmove` 를 써야 하는 이유이기도 합니다. 약속을 어기면 UB입니다.

4. container_of: 멤버 포인터에서 바깥 구조체 찾기

```
#include <stddef.h>
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

struct uart_dev { int id; struct list_node node; };
/* 리스트 순회 중 node 포인터만 알 때, 이를 담은 uart_dev를 얻는다 */
struct uart_dev *dev = container_of(node_ptr, struct uart_dev, node);
```

Linux 커널과 많은 RTOS의 "침습형(intrusive) 연결 리스트"의 핵심 기법으로, 노드를 malloc하지 않고 객체 안에 내장할 수 있게 해줍니다.

H. 면접 퀴즈: 포인터와 const

#	질문	답
1	<code>const char *p</code> , <code>char const *p</code> , <code>char *const p</code> 의 차이는?	앞의 둘은 동일(데이터가 const), 세 번째는 포인터가 const
2	<code>int *a[10]</code> 과 <code>int (*a)[10]</code> 의 차이는?	전자는 int 포인터 10개의 배열, 후자는 int 10개짜리 배열을 가리키는 포인터
3	<code>int (*fp)(int)</code> 와 <code>int *fp(int)</code> 의 차이는?	전자는 함수 포인터 변수, 후자는 <code>int *</code> 를 반환하는 함수 선언
4	변수가 const이면서 volatile일 수 있나?	예. 읽기 전용 HW 상태 레지스터
5	포인터 자체가 volatile일 수 있나?	예. <code>uint8_t *volatile p</code> (ISR이 포인터 값을 바꾸는 경우)
6	<code>typedef int *ip_t; const ip_t p;</code> 에서 p의 타입은?	<code>int *const p</code>
7	<code>char *s = "abc"; s[0] = 'x';</code> 는?	UB. 리터럴은 읽기 전용 영역
8	<code>void f(const int *p)</code> 안에서 <code>*p</code> 는 절대 안 바뀌나?	아니다. 다른 별칭 포인터, ISR, HW가 바꿀 수 있음
9	함수 안에서 <code>sizeof(arr)</code> 가 기대와 다른 이유는?	배열 매개변수는 포인터로 decay됨
10	<code>char **</code> 를 <code>const char **</code> 로 암묵 변환할 수 없는 이유는?	const 객체를 수정하는 경로가 생기기 때문 (II-C)

III. 하드웨어 직접 제어: 메모리 매핑과 비트 조작

임베디드 엔지니어의 핵심 역량은 소프트웨어 로직으로 물리적인 하드웨어를 직접 제어하는 것입니다. 이는 메모리 맵(Memory Map)과 비트 조작(Bit Manipulation)을 통해 이루어집니다.

A. 메모리 맵(Memory Map)과 Bare Metal 레지스터 접근

1. 메모리 매핑 I/O (MMIO)의 개념

MMIO에서 CPU는 RAM에 접근하는 것과 같은 방식(같은 주소 공간, 같은 load/store 명령어)으로 페리페럴(GPIO, UART, Timer 등)을 제어합니다. 각 페리페럴의 제어·상태·데이터 레지스터는 주소 공간의 특정 절대 주소에 매핑되어 있습니다. 데이터시트에 "UART3 데이터 레지스터(DR)가 0x40013804에 위치한다"고 적혀 있다면, 그 주소에 값을 쓰는 것으로 UART가 값을 전송합니다. (x86의 `in / out` 명령처럼 별도 I/O 주소 공간을 쓰는 방식은 Port-Mapped I/O라고 합니다.)

주의할 점은 MMIO 레지스터가 일반 메모리와 다르게 동작한다는 것입니다. **읽기만 해도 상태가 바뀐다(read-to-clear)** 레지스터 (예: UART DR을 읽으면 RXNE 플래그가 지워짐), **1을 써야 지워진다(write-1-to-clear, W1C)** 인터럽트 플래그 레지스터, **쓰기 전용** 레지스터가 혼잡합니다. 디버거 메모리 뷰로 이런 레지스터를 들여다보는 것만으로 동작이 바뀔 수 있습니다.

2. volatile 포인터를 사용한 하드웨어 레지스터 접근

- 데이터시트를 참조하여 레지스터의 절대 주소를 정의합니다. `#define UART3_DR_ADDR 0x40013804UL`
- 이 주소(정수)를 적절한 크기의 volatile 포인터로 캐스팅합니다.
- 포인터를 역참조하여 읽거나 씁니다.

```
/* 0x40013804 주소에 0x41 ('A')를 쓴다 */
*((volatile uint32_t *)UART3_DR_ADDR) = 0x41;

/* 0x40013804 주소에서 값을 읽어온다 */
uint32_t received_data = *((volatile uint32_t *)UART3_DR_ADDR);
```

레지스터 폭에 맞는 타입을 쓰는 것도 중요합니다. 8비트 레지스터에 `uint32_t *`로 접근하면 인접 레지스터까지 읽거나 쓰게 되고, 버스에 따라 폭이 맞지 않는 접근이 bus fault를 일으킵니다.

3. read_reg / write_reg 매크로 구현

```
#define WRITE_REG(addr, val) (*(volatile uint32_t *) (addr) = (val))
#define READ_REG(addr)      (*(volatile uint32_t *) (addr))

WRITE_REG(UART3_DR_ADDR, 0x41);
uint32_t data = READ_REG(UART3_DR_ADDR);
```

매크로와 포인터 변수의 성능 차이에 대해서는 II-D의 설명을 참고하세요. 핵심은 "주소가 컴파일 타임 상수인가"이며, `*const` 포인터 상수나 매크로 모두 같은 코드를 만듭니다.

구조체를 활용한 레지스터 맵 (실무 표준):

```
typedef struct {
    volatile uint32_t SR;    /* 0x00 Status Register */
    volatile uint32_t DR;    /* 0x04 Data Register */
    volatile uint32_t BRR;   /* 0x08 Baud Rate Register */
    volatile uint32_t CR1;   /* 0x0C Control Register 1 */
} UART_TypeDef;

#define UART3_BASE_ADDR 0x40013800UL
#define UART3 ((UART_TypeDef *)UART3_BASE_ADDR)

UART3->CR1 |= (1UL << 5);    /* RXNEIE 비트 세트 */
uint8_t data = (uint8_t)UART3->DR; /* 데이터 읽기 */
```

구조체 방식에서 면접관이 추가로 확인하는 포인트는 두 가지입니다. (1) 레지스터 사이에 빈 공간이 있으면 `uint32_t RESERVED0[2];` 같은 패딩 멤버로 오프셋을 맞춰야 하며, `_Static_assert(offsetof(UART_TypeDef, CR1) == 0x0C, "offset");` 로 컴파일 타임에 검증할 수 있습니다. (2) 모든 멤버가 같은 폭(`uint32_t`)이면 컴파일러가 패딩을 넣지 않지만, 폭이 섞이면 패딩이 생길 수 있어 주의해야 합니다(IV-B).

B. 비트 조작(Bit Manipulation) 테크닉

1. 비트 조작의 필요성

하드웨어 레지스터는 대부분 32비트(또는 16, 8비트) 워드 안에 여러 플래그(1비트)와 설정값(여러 비트)이 패킹되어 있습니다. 예를 들어 CR1의 5번 비트는 "수신 인터럽트 활성화", 10번 비트는 "송신 인터럽트 활성화"일 수 있습니다. 5번 비트만 켜고 나머지는 그대로 유지하려면 비트 연산자(`&`, `|`, `^`, `~`, `<<`, `>>`)가 필요합니다.

2. 필수 비트 연산 매크로

```
#define BIT(pos) (1UL << (pos))
#define SET_BIT(reg, pos) ((reg) |= BIT(pos))
#define CLEAR_BIT(reg, pos) ((reg) &= ~BIT(pos))
#define TOGGLE_BIT(reg, pos) ((reg) ^= BIT(pos))
#define CHECK_BIT(reg, pos) (((reg) >> (pos)) & 1UL) /* 항상 0 또는 1 반환 */
```

괄호가 필요한 진짜 이유: 인자에 연산자 우선순위가 `<<` 보다 낮은 식이 들어올 때 문제가 됩니다. 괄호 없는 `#define BAD_SET(REG, POS) (REG |= 1U << POS)` 에 `BAD_SET(reg, x & 0x7)` 을 넣으면 `reg |= (1U << x) & 0x7` 로 전개되어 전혀 다른 결과가 나옵니다. ($x = 0x13$ 일 때 직접 실행해 보면 괄호 없는 버전은 0x0, 올바른 버전은 0x8.) 참고로 draft-1의 `pin_num + 1` 예시는 `+` 가 `<<` 보다 우선순위가 높아 우연히 올바르게 계산되므로 적절한 반례가 아닙니다.

1 대신 1U / 1UL 을 쓰는 진짜 이유: `1 << 31` 은 signed int 오버플로로 UB입니다. 또한 폭 이상으로 시프트(`1UL << 32`, UL이 32비트일 때)도 UB이므로, 64비트 레지스터에는 `1ULL` 을 사용합니다.

CHECK_BIT 반환값 주의: `((reg) & (1U << (pos)))` 는 0 또는 `1 << pos` 를 반환합니다. 이 값을 `uint8_t` 변수에 저장하면 `pos ≥ 8` 일 때 잘려서 0이 되는 버그가 생기므로, 0/1을 반환하도록 만드는 편이 안전합니다.

3. 여러 비트로 된 필드 읽기/쓰기 (Read-Modify-Write)

실제 레지스터는 1비트 플래그보다 "3~5번 비트에 모드 값"처럼 여러 비트 필드가 많습니다.

```
#define FIELD_MASK(width, pos)      (((1UL << (width)) - 1UL) << (pos)) /* width < 32 */
#define FIELD_GET(reg, width, pos)  (((reg) & FIELD_MASK(width, pos)) >> (pos))
#define FIELD_SET(reg, width, pos, v) \
    ((reg) = ((reg) & ~FIELD_MASK(width, pos)) | (((uint32_t)(v) << (pos)) & FIELD_MASK(width, \
pos)))

/* 예: CR1의 [4:3] 2비트 필드를 0b10으로 설정 */
FIELD_SET(UART3->CR1, 2, 3, 0x2);
```

면접 심화 포인트:

- **Read-Modify-Write는 원자적이지 않다.** 메인 루프가 `reg |= bit` 을 수행하는 중간(읽기와 쓰기 사이)에 ISR이 같은 레지스터의 다른 비트를 바꾸면 ISR의 변경이 덮어써집니다. 크리티컬 섹션으로 보호하거나, 별도의 **SET/CLEAR 레지스터**(예: STM32 GPIO BSRR, NXP GPIO SET/CLR)를 사용하면 단일 쓰기로 원자적 제어가 가능합니다. ARM의 bit-banding(Cortex-M3/M4)도 같은 목적입니다.
- **W1C 레지스터에 |= 를 쓰면 안 된다.** 인터럽트 상태 레지스터가 write-1-to-clear일 때 `STATUS |= FLAG_A;` 는 읽어들인 값에 1로 켜져 있던 다른 플래그까지 전부 1을 써서 지워버립니다. 올바른 코드는 `STATUS = FLAG_A;` 입니다.
- **C 비트필드(bitfield) 구조체로 레지스터를 표현하는 것은 이식성이 없다.** 비트 할당 순서, 패딩, 접근 폭이 컴파일러 구현 정의이기 때문에 레지스터 정의에는 마스크/시프트 방식이 권장됩니다.

4. 알고리즘 1: 비트 카운팅 (Brian Kernighan's Algorithm)

질문: `int count_set_bits(uint32_t n)` 을 효율적으로 구현하시오.

```
int count_set_bits(uint32_t n) {
    int count = 0;
    while (n != 0) {
        n &= n - 1; /* 가장 오른쪽의 '1' 비트를 제거 */
        count++;
    }
    return count;
}
```

n & (n-1) 의 원리:

1. `n = 0b01011000` (88)이라고 가정합니다.
2. `n-1 = 0b01010111` (87). 1을 빼면 가장 오른쪽의 '1'이 '0'이 되고, 그 오른쪽의 '0'들이 모두 '1'이 됩니다.
3. 둘을 AND하면 가장 오른쪽 '1'과 그 아래 비트가 모두 0이 됩니다: `0b01011000 & 0b01010111 = 0b01010000`

4. 루프는 1인 비트 수(여기서는 3)만큼만 실행됩니다.

시간 복잡도는 $O(k)$, k 는 1인 비트의 개수입니다. 32비트를 모두 순회하는 방식은 $O(32)$ 입니다. 같은 원리로 `(n & (n - 1)) == 0` 은 "n이 2의 거듭제곱인가($n \neq 0$ 조건과 함께)"를 판별하는 단골 한 줄 코드입니다.

더 나은 답변: GCC/Clang의 `__builtin_popcount(n)` 은 CPU에 POPCNT 명령이 있으면 명령어 하나로, 없으면 분기 없는 비트 병렬 연산이나 룩업 테이블로 구현됩니다. 반복 호출이 많다면 8비트 룩업 테이블(256바이트, `static const` 로 플래시에 배치)도 좋은 절충안입니다. C23은 `stdc_count_ones` 를 표준화했습니다.

5. 알고리즘 2: 엔디안 스왑 (Endian Swapping)

질문: 32비트 값의 엔디안을 바꾸는 함수를 구현하시오. (예: `0x12345678` → `0x78563412`)

```
uint32_t swap_endian32(uint32_t v) {
    return ((v & 0xFF000000U) >> 24) |
        ((v & 0x00FF0000U) >> 8) |
        ((v & 0x0000FF00U) << 8) |
        ((v & 0x000000FFU) << 24);
}
```

컴파일러 내장 함수: GCC/Clang `__builtin_bswap32(v)`, MSVC `_byteswap_ulong(v)` 는 ARM `REV`, x86 `BSWAP` 명령 하나로 변환됩니다. 다만 최신 GCC/Clang은 위의 수동 구현 패턴도 인식해 `-O2` 에서 동일하게 `REV` 한 명령으로 만들어 주므로, "내장 함수가 압도적으로 빠르다"기보다는 **의도가 명확하고 최적화가 보장된다**는 것이 정확한 설명입니다. 네트워크 프로토콜에서는 `htonl` / `ntohl` 처럼 의미를 드러내는 함수를 쓰는 것이 좋습니다.

엔디안 판별 (단골 후속 질문):

```
static inline int is_little_endian(void) {
    const uint16_t probe = 0x0001;
    return *(const uint8_t *)&probe == 0x01; /* 하위 바이트가 먼저 저장되면 little */
}
```

문자 타입(`uint8_t` / `unsigned char`) 포인터로 다른 객체를 읽는 것은 strict aliasing 규칙의 예외로 허용되므로 합법입니다. union을 이용한 방법도 C에서는 허용되지만 C++에서는 UB입니다. ARM Cortex-M과 x86은 리틀 엔디안, 네트워크 바이트 순서는 빅 엔디안입니다.

IV. 메모리 구조와 정수 합정

"이 변수는 메모리 어디에 있나요?"와 "이 식의 결과는?"은 펌웨어 면접의 기본 질문입니다. 링커 스크립트와 startup 코드를 이해하고 있는지, C의 정수 변환 규칙이 만드는 버그를 아는지 확인합니다.

A. 펌웨어 메모리 레이아웃

섹션	위치	내용	예시
<code>.isr_vector</code>	Flash (주소 0 또는 VTOR)	초기 SP 값 + 예외/인터럽트 핸들러 주소	Reset_Handler
<code>.text</code>	Flash	기계어 코드	모든 함수
<code>.rodata</code>	Flash	const 전역/static, 문자열 리터럴	<code>static const uint8_t crc_table[256]</code>
<code>.data</code>	RAM (초기값은 Flash에 저장)	0이 아닌 초기값을 가진 전역/static	<code>int g_mode = 3;</code>
<code>.bss</code>	RAM	초기값 없음 또는 0인 전역/static	<code>static uint8_t rx_buf[256];</code>
heap	RAM (.bss 위, 위로 성장)	malloc 영역 (임베디드에서는 종종 사용 금지)	
stack	RAM (끝에서 아래로 성장)	지역 변수, 반환 주소, 레지스터 저장	ISR 진입 시 스택 프레임

리셋 후 main까지 (Cortex-M 기준):

- 하드웨어가 벡터 테이블 0번 항목을 SP에, 1번 항목(Reset_Handler 주소)을 PC에 로드합니다.
- Reset_Handler(startup 코드)가 `.data`의 초기값을 Flash에서 RAM으로 복사합니다.
- `.bss` 영역을 0으로 채웁니다. (그래서 초기화하지 않은 전역 변수가 0인 것이 보장됩니다.)
- `SystemInit()`으로 클럭 등을 설정하고, C++이라면 전역 객체 생성자(`__libc_init_array`)를 호출합니다.
- `main()`을 호출합니다.

면접 포인트: "`.data`와 `.bss`를 나누는 이유는?" → `.bss`는 초기값을 Flash에 저장할 필요가 없어 이미지 크기가 줄어듭니다. "스택 오버플로를 어떻게 감지하나?" → 스택 끝에 패턴(예: 0xDEADBEEF)을 채워두고 주기적으로 검사하는 stack painting, RTOS의 스택 워터마크(FreeRTOS `uxTaskGetStackHighWaterMark`), MPU 가드 영역, Cortex-M33의 `MSPLIM` 레지스터.

B. 구조체 패딩과 정렬

```
struct A { char c; int i; char d; };    /* sizeof = 12 : c,패딩3, i, d,패딩3 */
struct B { int i; char c; char d; };    /* sizeof = 8 : i, c, d, 패딩2 */
```

(위 값은 32/64비트 공통으로 int가 4바이트, 정렬 4인 환경에서 실측한 결과입니다.) 컴파일러는 각 멤버를 정렬 요구사항의 배수 위치에 놓고, 구조체 전체 크기를 가장 큰 정렬의 배수로 맞춥니다. 큰 멤버부터 선언하면 패딩이 줄어듭니다.

`__attribute__((packed))` 는 패딩을 없애지만, 정렬되지 않은 멤버 접근이 느려지거나 일부 CPU에서 fault를 일으키고, 멤버 주소를 포인터로 넘기면 II-G의 정렬 문제가 생깁니다. 통신 패킷은 packed 구조체로 직접 캐스팅하기보다 바이트 단위로 직렬화/역직렬화하는 것이 이식성 면에서 안전합니다.

C. 정수 승격과 부호 변환의 함정

코드	결과	이유
<code>uint8_t a = 200, b = 100; a + b</code>	300	<code>uint8_t</code> 는 연산 전에 int로 승격됨. 결과를 <code>uint8_t</code> 에 저장할 때 비로소 44로 잘림
<code>uint8_t m = 0x0F; ~m</code>	0xFFFFFFF0	승격 후 반전되어 상위 비트까지 1. <code>(uint8_t)~m</code> 으로 잘라야 0xF0
<code>int i = -1; unsigned u = 1; i < u</code>	0 (거짓)	비교 시 int가 unsigned로 변환되어 -1이 0xFFFFFFFF가 됨
<code>for (uint8_t i = 0; i < 300; i++)</code>	무한 루프	i는 255 다음 0으로 돌아가 영원히 300보다 작음
<code>uint16_t x = 60000; x * x</code>	UB 가능	승격된 int끼리 곱해 int 오버플로 (16비트 int MCU면 더 일찍)
<code>int32_t t = a - b;</code> (a, b는 uint32_t 타임스탬프)	음수가 큰 양수로	경과 시간 계산은 unsigned 뺄셈이 wrap-around로 오히려 올바름: <code>uint32_t elapsed = now - start;</code>

마지막 줄은 실무에서 중요합니다. 32비트 틱 카운터가 wrap-around되어도 `now - start` 를 **unsigned**로 계산하면 경과 시간이 올바르게 나옵니다(부록 A-1과 연결). unsigned 오버플로는 정의된 동작(모듈로 2^N)이고, signed 오버플로는 UB라는 차이를 설명할 수 있어야 합니다.

V. 인터럽트와 동시성

펌웨어 버그의 상당수는 "ISR과 메인 루프가 같은 데이터를 만질 때" 생깁니다. 면접관은 레이스 조건을 발견하고, 재현 시나리오를 말하고, 올바르게 막는 능력을 봅니다.

A. ISR 작성 원칙

- **짧게**: ISR은 플래그 세우기, 버퍼에 넣기, 세마포어 signal만 하고 실제 처리는 태스크/메인 루프로 넘깁니다(deferred processing, bottom half). 긴 ISR은 다른 인터럽트의 지연(latency)과 jitter를 키웁니다.
- **블로킹 금지**: `printf`, `malloc`, 뮤텍스 lock, 대기 루프, 긴 부동소수점 연산을 피합니다. `printf`는 재진입 불가이고, 느리고, 내부적으로 `malloc`이나 락을 쓸 수 있습니다.
- **공유 변수는 volatile + 원자적 접근**: I-A와 V-B 참조.
- **플래그 클리어 순서**: 인터럽트 소스 플래그를 제대로 지우지 않으면 ISR에서 나오자마자 다시 진입하는 인터럽트 폭주(interrupt storm)가 생깁니다. 레벨 트리거인지 에지 트리거인지 데이터시트에서 확인합니다.
- **ISR은 값을 반환하거나 인자를 받지 않는다**: `void Handler(void)` 형태이며, 필요한 컨텍스트는 전역(static) 상태로 주고받습니다.

B. 공유 데이터 보호: 크리티컬 섹션

```
extern volatile uint64_t g_ticks;    /* SysTick ISR에서 증가 */

uint64_t get_ticks(void) {
    uint32_t primask = __get_PRIMASK(); /* 현재 인터럽트 상태 저장 (CMSIS) */
    __disable_irq();
    uint64_t t = g_ticks;                /* 32비트 CPU에서 2번의 LDR → 보호 필요 */
    __set_PRIMASK(primask);              /* 원래 상태로 복원 (중첩 호출에도 안전) */
    return t;
}
```

`__enable_irq()` 로 무조건 켜지 않고 **이전 상태를 복원**하는 이유는, 이 함수가 이미 인터럽트가 꺼진 구간에서 호출되었을 때 의도치 않게 인터럽트를 켜버리지 않기 위해서입니다. 크리티컬 섹션은 최대한 짧게 유지하고, RTOS에서는 `taskENTER_CRITICAL()` 같은 API를 사용합니다. 인터럽트를 끄지 않는 대안으로는 부록 A-1의 "다시 읽기(retry)" 패턴과 V-C의 SPSC 링버퍼가 있습니다.

C. 락 없는 SPSC 원형 버퍼 (ISR → main)

ISR 하나가 쓰고 메인 루프 하나가 읽는 경우(Single-Producer Single-Consumer), **각 인덱스를 한쪽만 쓰도록** 설계하면 락이 필요 없습니다.

```

#define RB_SIZE 256u                                /* 반드시 2의 거듭제곱 */

typedef struct {
    uint8_t buf[RB_SIZE];
    volatile uint32_t head;                          /* producer(ISR)만 수정 */
    volatile uint32_t tail;                          /* consumer(main)만 수정 */
} ringbuf_t;

/* ISR에서 호출 */
bool rb_put(ringbuf_t *rb, uint8_t v) {
    uint32_t h = rb->head;
    if ((uint32_t)(h - rb->tail) == RB_SIZE) {
        return false;                                /* full: 데이터 드롭 (카운터로 기록 권장) */
    }
    rb->buf[h & (RB_SIZE - 1u)] = v;                 /* 1) 데이터를 먼저 쓰고 */
    rb->head = h + 1u;                                /* 2) 인덱스를 나중에 공개 */
    return true;
}

/* main에서 호출 */
bool rb_get(ringbuf_t *rb, uint8_t *out) {
    uint32_t t = rb->tail;
    if (rb->head == t) {
        return false;                                /* empty */
    }
    *out = rb->buf[t & (RB_SIZE - 1u)];
    rb->tail = t + 1u;
    return true;
}

```

왜 안전한가:

- head는 ISR만, tail은 main만 씁니다. 상대방 인덱스는 **읽기만** 하므로 read-modify-write 경합이 없습니다.
- 인덱스를 마스킹하지 않고 계속 증가시키면 `head - tail` 이 unsigned wrap-around 덕분에 항상 저장된 개수가 됩니다. 그래서 N-1 기법 없이 **N칸을 모두** 사용하고도 full/empty를 구분합니다.
- 데이터를 쓴 **다음에** head를 갱신하므로, consumer는 완성된 데이터만 봅니다.
- 전제 조건: 32비트 인덱스의 읽기/쓰기가 단일 명령으로 원자적인 32비트 MCU, 그리고 정확히 하나의 producer와 하나의 consumer. 멀티코어에서는 C11 atomic 등으로 가시성을 보장해야 합니다.

VII-B의 `count` 변수 방식이 ISR 환경에서 위험한 이유도 여기서 설명됩니다. `count++` (ISR)와 `count--` (main)는 **양쪽이 같은 변수를 read-modify-write**하므로 보호 없이는 값이 틀어집니다.

D. 재진입성(Reentrancy)과 스레드 안전성

- **재진입 가능(reentrant) 함수**: 실행 중에 인터럽트되어 같은 함수가 다시 호출되어도 올바르게 동작합니다. 조건은 static/전역 상태를 수정하지 않고, 재진입 불가 함수를 호출하지 않고, 공유 하드웨어를 보호 없이 만지지 않는 것입니다.
- **재진입 불가의 전형**: static 지역 변수로 결과를 반환하는 함수(`strtok` , `asctime`), 내부 버퍼를 쓰는 `printf` , 전역 `errno` 에 의존하는 코드.
- ISR과 태스크가 공통으로 호출하는 유틸리티 함수는 반드시 재진입 가능하게 작성해야 합니다.

E. 워치독(Watchdog)

워치독 타이머는 소프트웨어가 정해진 시간 안에 "kick(feed)"하지 않으면 시스템을 리셋합니다. 면접 포인트는 **어디서 kick하는가**입니다. 타이머 ISR에서 kick하면 메인 로직이 멈춰도 ISR은 계속 돌기 때문에 워치독이 무의미해집니다. 메인 루프 (또는 각 태스크가 살아있음을 보고한 뒤 감시 태스크)에서 kick해야 합니다. 리셋 후에는 리셋 원인 레지스터를 읽어 워치독 리셋인지 기록하는 것이 디버깅에 중요합니다.

VI. RTOS 핵심 동기화 기법: 뮉텍스와 세마포어

실시간 운영체제(RTOS) 환경에서는 여러 태스크가 동시에 실행되며 공유 자원에 접근합니다. 데이터 오염이나 데드락을 막기 위한 동기화 프리미티브 중 뮉텍스와 세마포어가 가장 기본적이고 중요합니다.

A. 뮉텍스(Mutex) vs. 세마포어(Semaphore): 핵심 차이

"뮉텍스는 카운트가 1인 세마포어(바이너리 세마포어)다"라는 말은 널리 퍼진 오해입니다. 둘은 사용 **의도**와 **구현**이 근본적으로 다릅니다.

1. 핵심 차이 1: 용도 (Intent)

- **뮉텍스 (MUTual EXclusion)**: 상호 배제가 유일한 목적입니다. 여러 태스크가 동시에 접근하면 안 되는 공유 자원(I2C 버스, UART 전송 버퍼, 전역 설정)을 보호합니다. "화장실 열쇠" 비유처럼 한 번에 한 태스크만 락을 가질 수 있습니다.
- **세마포어**: 주된 목적은 **시그널링**입니다. 태스크 간 실행 순서를 맞추거나(Producer-Consumer), 사용 가능한 자원의 수를 셉니다(카운팅 세마포어, 예: 버퍼 풀의 남은 블록 수).

비유: 뮉텍스는 같은 역할의 두 태스크가 제3의 자원을 두고 순서를 지키는 것이고, 세마포어는 역할이 다른 두 태스크 중 하나(Producer)가 다른 하나(Consumer)에게 "데이터 준비 완료" 신호를 보내는 것입니다.

2. 핵심 차이 2: 소유권 (Ownership)

- 뮉텍스: 소유권이 있습니다. lock(take)한 태스크만 unlock(give)할 수 있고, 다른 태스크가 해제를 시도하면 대부분의 RTOS가 에러를 반환합니다. 소유권이 있기 때문에 **우선순위 상속**과 **재귀 락(recursive mutex)**이 가능합니다.
- 세마포어: 소유권이 없습니다. 태스크 A가 signal(give)하고 태스크 B가 wait(take)하는 것이 정상 사용법입니다.

3. 핵심 차이 3: ISR에서의 사용

- 뮉텍스: lock은 블로킹될 수 있고, 소유자가 "태스크"여야 하므로 ISR에서 사용하면 안 됩니다. (FreeRTOS 문서도 뮉텍스를 ISR에서 사용하지 말 것을 명시합니다.)
- 세마포어: give는 non-blocking이라 ISR에서 사용하는 것이 매우 일반적입니다. 단, RTOS마다 **ISR 전용 API**를 써야 합니다.

```
/* FreeRTOS: UART RX ISR에서 처리 태스크 깨우기 */
void USART3_IRQHandler(void) {
    BaseType_t woken = pdFALSE;
    /* ... 수신 바이트를 링버퍼에 넣고 플래그 클리어 ... */
    xSemaphoreGiveFromISR(g_rx_sem, &woken);
    portYIELD_FROM_ISR(woken); /* 더 높은 우선순위 태스크가 깨어났으면 ISR 종료 직후 전환 */
}
```

일반 API(`xSemaphoreGive`)를 ISR에서 호출하면 스케줄러 내부 자료구조가 깨질 수 있습니다. `FromISR` API와 `portYIELD_FROM_ISR`의 역할을 설명할 수 있으면 실무 경험을 보여줄 수 있습니다. 태스크 하나만 깨우는 용도라면 세마포어보다 가벼운 **task notification**(`xTaskNotifyFromISR`)도 좋은 선택입니다.

4. 비교 요약표

표 2: 뮤텍스(Mutex) vs. 세마포어(Semaphore) 핵심 비교

속성	뮤텍스 (Mutex)	세마포어 (Semaphore)
핵심 용도	상호 배제 (Mutual Exclusion)	시그널링 / 자원 카운팅
소유권	있음. 획득한 태스크만 해제 가능	없음. 누구나 give/take 가능
연산	lock, unlock	wait (P, take), signal (V, give)
종류	일반, 재귀(recursive)	바이너리(0/1), 카운팅(0...N)
ISR 사용	불가	give는 ISR 전용 API로 가능
우선순위 역전 대응	우선순위 상속 지원	지원 안 함

B. 우선순위 역전(Priority Inversion) 문제와 해결

"뮤텍스와 세마포어의 차이"를 묻는 진짜 의도는 "우선순위 역전"을 이해하고, 뮤텍스가 이를 어떻게 완화하는지 아는지 확인하는 것입니다.

1. 문제 정의: 우선순위 역전

높은 우선순위 태스크(H)가 낮은 우선순위 태스크(L) 때문에, 그리고 결과적으로 **관련 없는 중간 우선순위 태스크(M)** 때문에 무한정 실행되지 못하는 상황입니다.

1. Task L이 Mutex A를 획득하고 임계 영역을 실행합니다.
2. Task H가 준비되어 L을 선점합니다.
3. H가 Mutex A를 요청하지만 L이 소유 중이므로 H는 블록됩니다.
4. L이 다시 실행되어야 Mutex A가 해제되고 H가 진행할 수 있습니다.
5. 이때 Mutex A와 관련 없는 Task M이 준비 상태가 됩니다.
6. 스케줄러는 L보다 우선순위가 높은 M을 실행합니다.
7. 결과: M이 실행되는 동안 L은 멈추고, H는 L을 기다립니다. **가장 높은 우선순위인 H가 M에게 막히며, M이 오래 돌수록 H의 대기 시간은 한없이 길어집니다(unbounded).**

실제 사례로 1997년 화성 탐사선 **Mars Pathfinder**가 반복적으로 리셋된 원인이 VxWorks에서의 우선순위 역전이었고, 지상에서 뮤텍스의 우선순위 상속 옵션을 켜는 패치로 해결되었습니다. 면접에서 이 사례를 언급하면 문제의 심각성을 효과적으로 전달할 수 있습니다.

2. 해결 방안 1: 우선순위 상속 (Priority Inheritance)

1. H가 L이 소유한 뮤텍스를 기다리며 블록되는 순간, RTOS가 이를 감지합니다.
2. L의 우선순위를 H의 우선순위로 일시적으로 올립니다.
3. 이제 L은 M보다 높으므로 즉시 실행되어 임계 영역을 끝냅니다.
4. 뮤텍스를 해제하는 순간 L은 원래 우선순위로 돌아갑니다.
5. H가 뮤텍스를 획득해 실행을 재개합니다.

한계: 역전 시간을 "L의 임계 영역 길이"로 **한정(bounded)**할 뿐 없애지는 못하며, 중첩 락에서는 상속이 연쇄적으로 일어나 분석이 복잡해집니다.

3. 해결 방안 2: 우선순위 상한 (Priority Ceiling)

뮤텍스마다 "이 뮤텍스를 사용할 수 있는 태스크 중 최고 우선순위(ceiling)"를 미리 정해두고, 뮤텍스를 획득하는 즉시 소유 태스크의 우선순위를 ceiling으로 올립니다. 역전을 사전에 차단하고, 락 순서가 강제되는 효과로 **데드락도 방지**됩니다. POSIX의 `PTHREAD_PRIO_PROTECT` 가 이 방식이며, 안전 필수 시스템에서 선호됩니다. 대신 모든 사용 태스크를 미리 알아야 합니다.

4. 결론과 데드락 보너스

상호 배제가 목적이라면 우선순위 상속이 구현된 **뮤텍스**를 사용해야 합니다. 바이너리 세마포어를 상호 배제용으로 쓰면 상속이 동작하지 않아 역전에 무방비로 노출됩니다. 후속 질문으로 자주 나오는 **데드락**은 네 조건(상호 배제, 점유 대기, 비선점, 순환 대기)이 모두 성립할 때 발생하며, 실무에서 가장 쉬운 예방법은 **모든 태스크가 락을 항상 같은 순서로 획득**하게 하는 것(순환 대기 제거)과 **타임아웃 있는 lock**입니다.

VII. 임베디드 시스템을 위한 핵심 자료구조 구현

임베디드 시스템은 제한된 메모리와 실시간 제약 조건 하에서 동작하므로, 자료구조도 데스크톱과는 다른 기준으로 선택하고 구현해야 합니다.

A. 단일 연결 리스트(Singly Linked List)

1. 개념 및 C언어 구현

```
struct Node {  
    void *data;          /* 범용 구현. 임베디드에서는 값을 직접 담는 경우가 많음 */  
    struct Node *next;  
};
```

핵심 연산:

- `insert_first(struct Node **head, struct Node *n)` : $O(1)$. `n->next = *head; *head = n;`
- `insert_last(struct Node **head, struct Node *n)` : $O(N)$. 끝까지 순회 후 연결. tail 포인터를 따로 유지하면 $O(1)$.
- `remove(struct Node **head, const struct Node *target)` : $O(N)$. 이전 노드를 찾아 `prev->next = target->next`. II-G-1의 이중 포인터 기법을 쓰면 head 삭제 분기 없이 한 번에 처리할 수 있습니다.

`struct Node **head` 를 받는 이유는 head 자체(호출자의 포인터 변수)를 바꿔야 하기 때문입니다. 단일 포인터로 받으면 함수 안에서 바꾼 head가 호출자에게 반영되지 않습니다. 이 "왜 이중 포인터인가"는 거의 반드시 묻는 질문입니다.

2. 심화 질문: "malloc/free를 사용하지 않고 구현하시오."

질문의 의도: 안전필수 시스템(항공, 의료, 자동차; MISRA C, DO-178C 환경)에서는 초기화 이후 힙 사용을 금지하는 경우가 많습니다.

1. **메모리 파편화:** 할당/해제를 반복하면 총량은 충분해도 연속된 큰 블록을 할당할 수 없게 됩니다. 수개월 연속 동작하는 장비에서 치명적입니다.
2. **비결정적 실행 시간:** malloc이 빈 블록을 찾는 시간을 예측할 수 없어 실시간 보장이 깨집니다.
3. **실패 처리:** malloc이 NULL을 반환했을 때 펌웨어가 할 수 있는 합리적인 복구가 거의 없습니다.

해결책: 정적 메모리 풀 (Static Memory Pool)

```

#define MAX_NODES 32

static struct Node g_node_pool[MAX_NODES];
static struct Node *g_free_list_head;

void pool_init(void) {
    g_free_list_head = NULL;
    for (size_t i = 0; i < MAX_NODES; i++) {
        g_node_pool[i].next = g_free_list_head; /* 모든 노드를 free list에 연결 */
        g_free_list_head = &g_node_pool[i];
    }
}

struct Node *node_alloc(void) { /* O(1) */
    struct Node *node = g_free_list_head;
    if (node != NULL) {
        g_free_list_head = node->next;
    }
    return node; /* 풀 고갈 시 NULL */
}

void node_free(struct Node *node) { /* O(1) */
    node->next = g_free_list_head;
    g_free_list_head = node;
}

```

장점: 파편화 없음, O(1) 결정적 시간, 헤더 오버헤드 없음, 최대 사용량이 컴파일 타임에 확정. 후속 질문 대비 포인트:

- ISR과 태스크가 함께 alloc/free한다면 free list 조작을 크리티컬 섹션으로 보호해야 합니다.
- 크기가 다양한 객체가 필요하면 크기별 풀(16/64/256바이트 블록)을 여러 개 둡니다. FreeRTOS의 `heap_1` (해제 없음)~ `heap_4` (병합) 선택지도 같은 고민의 결과입니다.
- `node_free` 에 풀 범위 밖 포인터나 이중 해제가 들어오는 것을 막으려면 주소 범위 검사와 사용 중 플래그를 둘 수 있습니다.

B. 원형 버퍼(Circular Buffer / Ring Buffer)

1. 개념

고정 크기 배열의 끝과 시작을 논리적으로 이어 붙인 FIFO 큐입니다. UART, SPI, I2S, DMA, 오디오 스트림 버퍼링에 쓰이며, ISR은 빠르게 넣고(Producer) 메인 루프는 여유롭게 꺼냅니다(Consumer).

```
typedef struct {
    uint8_t *buffer;
    size_t head;      /* 다음에 쓸 위치 (write index) */
    size_t tail;      /* 다음에 읽을 위치 (read index) */
    size_t capacity;
} circular_buffer_t;
```

- put: `buffer[head] = data; head = (head + 1) % capacity;`
- get: `data = buffer[tail]; tail = (tail + 1) % capacity;`

% 는 하드웨어 나눗셈기가 없는 MCU(Cortex-M0 등)에서 느리므로, `if (++head == capacity) head = 0;` 또는 `capacity`를 2의 거듭제곱으로 두고 `& (capacity - 1)` 마스크를 사용합니다.

2. 난제: Full과 Empty 구별

기본 연산만으로는 비었을 때와 가득 찼을 때 모두 `head == tail` 이 되어 구별할 수 없습니다. 표준 해법은 네 가지입니다.

방법	is_empty	is_full	장점	단점
1. 한 칸 비우기 (N-1)	<code>head == tail</code>	<code>(head + 1) % N == tail</code>	추가 변수 없음, SPSC 락프리 가능	1칸 낭비
2. count 변수	<code>count == 0</code>	<code>count == N</code>	가장 직관적	ISR/main이 모두 count를 수정 → 보호 필요
3. full 플래그	<code>head == tail && !full</code>	<code>full</code>	1칸 낭비 없음	플래그도 양쪽이 수정 → 보호 필요
4. 마스크 안 한 인덱스	<code>head == tail</code>	<code>head - tail == N</code>	N칸 모두 사용 + SPSC 락프리	N이 2의 거듭제곱이어야 함

면접에서는 "ISR이 put하고 main이 get한다면 어떤 방법이 좋은가?"로 이어지는 경우가 많습니다. 답은 **1번 또는 4번**이며 이유와 코드는 V-C에 있습니다.

3. 연결 리스트 vs. 원형 버퍼

- **원형 버퍼**: 정적 할당, 연속 메모리로 캐시 효율적, $O(1)$ enqueue/dequeue. 단점은 고정 크기. 바이트 스트림(UART RX), 오디오 샘플, 로그 버퍼에 적합합니다.
- **연결 리스트**: 크기가 유동적이고 중간 삽입/삭제가 $O(1)$ (위치를 알 때). 단점은 노드 할당 필요(또는 정적 풀), 캐시 비효율, 노드당 포인터 오버헤드. 가변 길이 패킷 큐, RTOS의 태스크 대기 리스트·타이머 리스트에 적합합니다. RTOS 내부 리스트는 대부분 TCB 안에 노드를 내장하는 침습형 리스트(II-G-4)로 malloc 없이 구현됩니다.

VIII. C/C++ 코드 분석: 숨겨진 동작과 고급 패턴

면접관은 짧은 코드 조각을 보여주고 "출력은?" 또는 "문제점은?"을 묻습니다. 언어 표준의 미묘한 부분과 함정을 이해하는지 평가하기 위함입니다.

A. C 코드: printf와 정의되지 않은 동작 (Undefined Behavior)

1. 문제 사례

```
int i = 5;
printf("%d %d\n", i++, i);
```

"5 5" 또는 "5 6"이라고 답하는 순간 함정에 빠집니다. 정답은 "C에서 이 코드는 정의되지 않은 동작(UB)이므로 어떤 결과가 나올 수 있고, 결과를 논하는 것 자체가 의미 없다. 두 문장으로 나눠야 한다"입니다.

2. UB vs. Unspecified vs. Implementation-defined

분류	의미	예
Implementation-defined	구현이 선택하되 문서화해야 함	<code>sizeof(int)</code> , 음수의 오른쪽 시프트 결과, <code>char</code> 의 부호
Unspecified	몇 가지 중 하나를 고르며 문서화 의무 없음	함수 인자의 평가 순서
Undefined	표준이 아무 요구도 하지 않음	signed 오버플로, 널 역참조, 시퀀스 포인트 사이의 이중 수정

- **인자 평가 순서 (Unspecified):** C 표준은 `i++`와 `i` 중 무엇을 먼저 평가할지 정하지 않습니다. draft-10에서 설명한 "스택 푸시 순서" 모델은 정확하지 않습니다. ARM(AAPCS)이나 x86-64에서는 앞쪽 인자가 레지스터로 전달되며, 평가 순서와 전달 방식은 별개의 문제입니다.
- **시퀀스 포인트 위반 (Undefined):** 시퀀스 포인트는 그 이전 식의 부작용이 모두 완료되었음이 보장되는 지점입니다. C에서는 완전식(full expression)의 끝(`;` 등), `&&`, `||`, `?:`의 첫 피연산자 평가 후, **coma 연산자**, 그리고 **함수 호출 직전(인자와 함수 지정자 평가가 끝난 뒤)**이 해당합니다. 두 시퀀스 포인트 사이에서 같은 객체를 (1) 두 번 수정하거나 (2) 수정하면서 그 값을 다른 목적으로 읽으면 UB입니다. `printf` 인자를 구분하는 쉼표는 **coma 연산자가 아니라 구분자**이므로 시퀀스 포인트가 아니고, `i++` (수정)와 `i` (읽기)가 한 구간에 있어 UB입니다.
- 참고: C11부터 표준 용어는 "sequenced/unsequenced"로 바뀌었고, C++17은 함수 인자들이 서로 끼어들어(interleave) 평가되지 않도록 규칙을 바꿔 이 예제가 C++17에서는 UB가 아닌 unspecified(`5 5` 또는 `5 6`)가 됩니다. C에서는 여전히 UB입니다.
- UB가 무서운 이유: 컴파일러는 "UB는 발생하지 않는다"고 가정하고 최적화합니다. 예를 들어 `if (x + 1 < x)`로 signed 오버플로를 검사하는 코드는 조건이 항상 거짓이라고 판단되어 **검사 자체가 삭제될** 수 있습니다. `fsanitize=undefined` (UBSan)로 개발 중에 잡는 습관이 중요합니다.

3. 연관 함정: 연산자 우선순위와 증감 연산자

- `*ptr++`: 후위 `++`가 `*`보다 우선순위가 높아 `*(ptr++)`입니다. 식의 값은 **증가 전 주소**를 역참조한 값이고, `ptr`은 다음 원소를 가리키게 됩니다. `while (n--) *dst++ = *src++;`가 대표적인 복사 관용구입니다.

- `*++ptr` : `*(&ptr)` . ptr을 먼저 증가시키고 새 주소를 역참조합니다.
- `++*ptr` : `++(&ptr)` . ptr이 가리키는 값을 증가시키고 주소는 그대로입니다.
- `(*ptr)++` : 값을 증가시키되 식의 값은 증가 전 값입니다. 괄호를 빼먹어 `*ptr++` 로 쓰면 포인터가 움직이는 버그가 됩니다.

B. C++ 코드: 객체 생명주기와 리소스 관리

cout이 들어간 C++ 상속 코드의 출력을 묻는 질문은 **생성자와 소멸자가 언제, 어떤 순서로 호출되는가**를 묻는 것입니다.

1. 객체 생성 순서: "기반에서 파생으로, 선언 순서대로"

1. 기본 클래스 생성자 (다중 상속이면 상속 선언 순서대로; 가상 기반 클래스가 가장 먼저)
2. 멤버 변수 초기화: **클래스 안에 선언된 순서대로**. 멤버 초기화 리스트에 적은 순서가 아닙니다. 순서가 다르면 `-Wreorder` 경고가 나며, 앞 멤버가 아직 초기화되지 않은 뒤 멤버를 사용하는 버그의 원인이 됩니다.
3. 생성자 본문 실행

2. 객체 소멸 순서: 생성의 완벽한 역순

1. 소멸자 본문 실행
2. 멤버 소멸 (선언의 역순)
3. 기본 클래스 소멸자 (상속 선언의 역순)

3. 반드시 나오는 후속 질문: 가상 소멸자

```
struct Base { ~Base() { /* ... */ } };           /* virtual 없음 */
struct Derived : Base { std::vector<int> v; };

Base *p = new Derived;
delete p;    /* UB: Derived의 소멸자가 호출되지 않을 수 있음 → v 누수 등 */
```

다형적으로 사용할(기반 포인터로 삭제할) 클래스의 소멸자는 반드시 `virtual` 이어야 합니다. 또 하나의 함정은 **생성자나 소멸자 안에서 가상 함수를 호출하면 파생 클래스 버전이 아니라 현재 클래스 버전이 호출된다는 점**입니다. 생성 중에는 파생 부분이 아직 만들어지지 않았기 때문입니다.

4. 핵심 원리: RAII (Resource Acquisition Is Initialization)

C++ 객체 생명주기 질문은 RAII로 이어집니다. 질문의 의도는 C++를 "더 나은 C"로만 쓰는지, 스코프 기반 리소스 관리를 이해하는지 확인하는 것입니다.

- 리소스(메모리, 파일, 소켓, 뮤텍스, 인터럽트 비활성 구간) 획득을 생성자에 맡깁니다.
- 해제를 소멸자에 맡깁니다.
- 객체를 스택(지역 변수)에 선언합니다.

C 코드의 문제점:

```

void c_style_func(void) {
    lock_mutex(&my_mutex);      /* (1) 락 획득 */
    do_stuff();
    if (error) {
        /* unlock_mutex(&my_mutex);   (2) 이 줄을 잊으면? */
        return;                  /* 락을 쥔 채 반환 → 다음 lock에서 데드락 */
    }
    do_more_stuff();
    unlock_mutex(&my_mutex);     /* (3) 정상 해제 */
}

```

C에서는 모든 반환 경로에서 해제를 수동으로 보장해야 하며, 흔히 `goto cleanup;` 패턴으로 한 곳에 모읍니다(Linux 커널 스타일).

RAII 솔루션 (`std::lock_guard` 와 같은 원리):

```

class MutexLocker {
public:
    explicit MutexLocker(MutexType *m) : p_mutex(m) { lock_mutex(p_mutex); }
    ~MutexLocker() { unlock_mutex(p_mutex); }
    MutexLocker(const MutexLocker &) = delete;          /* 복사 금지: 이중 unlock 방지 */
    MutexLocker &operator=(const MutexLocker &) = delete;
private:
    MutexType *p_mutex;
};

void cpp_style_func() {
    MutexLocker lock(&my_mutex); /* 생성자에서 락 획득 */
    do_stuff();
    if (error) {
        return;                  /* 스코프를 벗어나며 소멸자 자동 호출 → 락 해제 */
    }
    do_more_stuff();
}
/* 정상 종료 시에도 자동 해제 */

```

복사 금지(= `delete`)는 RAII 클래스 작성 시 면접관이 꼭 확인하는 디테일입니다. 복사가 허용되면 두 객체가 같은 뮤텁스를 두 번 해제합니다.

결론: RAII는 스코프를 벗어나는 모든 경로(return, break, continue, 예외)에서 소멸자 호출이 언어 차원에서 보장되므로 리소스 누수와 데드락을 구조적으로 막습니다.

5. 임베디드 C++에서 주의할 점

- 많은 펌웨어 프로젝트는 코드 크기와 결정성을 위해 `-fno-exceptions -fno-rtti` 로 빌드합니다. RAII는 예외 없이도 완전히 동작하므로 여전히 유용합니다.

- 동적 할당을 숨기는 컨테이너(`std::vector`, `std::string`, `std::function`)는 힙을 사용합니다. 고정 용량 컨테이너(`std::array`, ETL 라이브러리)를 선호합니다.
- 가상 함수는 vtable 포인터(객체당 포인터 1개)와 간접 호출 비용이 있지만 대부분 무시할 수준입니다. ISR 경로처럼 시간이 중요한 곳에서는 템플릿 기반 정적 다형성(CRTP)을 쓰기도 합니다.
- 전역 객체 생성자는 main 이전 startup 코드에서 실행되므로, 하드웨어 초기화 전에 페리페럴을 만지는 생성자는 문제가 됩니다(static initialization order fiasco와 함께 자주 나오는 이야기).

IX. 통신 프로토콜 기초: UART, I2C, SPI

펌웨어 면접에서는 "세 프로토콜을 비교하라"와 "버스가 멈췄을 때 어떻게 디버깅하나"가 거의 반드시 나옵니다. 스코프와 로직 분석기로 파형을 본 경험을 구체적으로 이야기할 수 있으면 가장 강력한 답이 됩니다.

A. 비교표

항목	UART	I2C	SPI
신호선	TX, RX (+GND)	SDA, SCL (+GND)	SCLK, MOSI(SDO), MISO(SDI), CS 디바이스당 1개
클럭	없음 (비동기, 양쪽이 baud rate 합의)	있음 (마스터가 생성, 슬레이브가 늘릴 수 있음)	있음 (마스터만 생성)
토폴로지	1:1	멀티 마스터 / 멀티 슬레이브 버스 (7비트 주소)	1 마스터 : N 슬레이브 (CS로 선택)
듀플렉스	전이중	반이중	전이중
속도	보통 9600 ~ 수 Mbps	100k / 400k / 1M / 3.4M bps	수 ~ 수십 Mbps
응답 확인	없음 (패리티 선택)	바이트마다 ACK/NACK	없음 (상위 프로토콜에서 처리)
출력 구조	Push-pull	Open-drain + 풀업 저항	Push-pull
대표 용도	디버그 콘솔, GPS, 모뎀, BLE 모듈	센서, EEPROM, PMIC, 저속 설정	플래시, 디스플레이, 고속 ADC, IMU

B. 자주 나오는 심화 질문

- **I2C는 왜 풀업 저항이 필요한가?** 모든 디바이스가 선을 Low로 당기기만(open-drain) 하고 High는 풀업이 만들기 때문입니다. 이 구조 덕분에 여러 마스터가 동시에 구동해도 단락이 없고, 중재(arbitration)와 clock stretching이 가능합니다. 풀업 값이 너무 크면 버스 커패시턴스 때문에 상승 시간이 길어져 고속 모드에서 실패하고, 너무 작으면 Low 구동 전류와 소비 전력이 커집니다.
- **Clock stretching:** 슬레이브가 처리 시간이 필요할 때 SCL을 Low로 붙잡아 마스터를 기다리게 하는 기능입니다. 마스터 드라이버가 이를 지원하지 않거나 타임아웃이 없으면 통신 오류나 hang이 생깁니다.
- **I2C 버스가 SDA Low로 멈췄다 (bus hang):** 전송 도중 마스터가 리셋되면 슬레이브는 여전히 비트를 보내는 중이라 SDA를 Low로 붙잡고 있을 수 있습니다. 표준 복구 절차는 SCL을 GPIO로 전환해 **최대 9번 클럭을 토글**하면서 SDA가 High로 풀리는지 확인하고, 이후 STOP 조건을 만드는 것입니다.
- **NACK의 의미:** 주소 단계 NACK은 디바이스 없음·주소 오류·전원/리셋 상태를, 데이터 단계 NACK은 디바이스가 바쁘거나 레지스터가 잘못된 경우를 뜻합니다. 로직 분석기로 주소 바이트(7비트 주소를 1비트 시프트한 값인지 혼동 여부 포함)를 먼저 확인합니다.
- **SPI 모드(CPOL/CPHA):** CPOL은 idle 시 클럭 레벨, CPHA는 첫 번째(0)와 두 번째(1) 에지 중 어디서 샘플링할지를 정합니다. 조합으로 Mode 0~3이 있고, 마스터와 슬레이브의 모드가 다르면 **데이터가 1비트 밀리거나 간헐적으로 깨지는** 증상이

나타납니다. 가장 흔한 것은 Mode 0입니다.

- **UART 오류 종류:** framing error(스톱 비트 위치에 1이 아님 → baud rate 불일치나 노이즈), overrun error(이전 바이트를 읽기 전에 다음 바이트 도착 → ISR 지연, DMA나 링버퍼 필요), parity error. 클럭 오차가 누적되므로 양쪽 오차 합이 약 $\pm 2\sim 3\%$ 안이어야 안정적으로 통신합니다.
- **CAN은 어떻게 다른가?** 차동 2선(CAN_H/CAN_L), 멀티 마스터, 메시지 ID 기반 **비파괴 중재**(dominant 0이 이기므로 ID가 작을수록 우선순위가 높음), 프레임마다 CRC와 ACK 슬롯, 에러 카운터 기반 bus-off 상태가 있습니다. Classic CAN은 최대 1Mbps·8바이트, CAN FD는 데이터 구간 고속화와 최대 64바이트 페이로드를 지원합니다.

X. 알고리즘 문제 해결: LeetCode 패턴과 C 구현 관점

임베디드 엔지니어에게도 알고리즘 문제 해결 능력은 필수입니다. Python을 허용하는 면접도 많지만, 펌웨어 포지션은 **C로 풀기를** 요구하거나 **C로 풀면 가산점**인 경우가 많으므로 두 언어 관점을 모두 준비합니다.

A. LeetCode (Easy/Medium) 핵심 패턴

1. **해시 맵**: 존재 여부·빈도수를 $O(1)$ 에 조회. 예: Two Sum, Contains Duplicate, Valid Anagram, Group Anagrams. C에서는 해시 테이블이 없으므로 **키 범위가 작으면 배열을 카운터로** (`int count[256]`), 크면 오픈 어드레싱 테이블을 직접 구현할 수 있어야 합니다.
2. **투 포인터**: Slow/Fast(Linked List Cycle, 중간 노드 찾기), Left/Right(Two Sum II, 부록 A-5 Palindrome).
3. **슬라이딩 윈도우**: 연속 부분 배열/문자열 조건. 예: Longest Substring Without Repeating Characters. 임베디드의 이동 평균 필터와 같은 사고방식입니다.
4. **이진 탐색**: 정렬 데이터에서 $O(\log N)$. 예: First Bad Version. `mid = lo + (hi - lo) / 2` 로 오버플로를 피하는 것까지 확인받습니다.
5. **스택 / 힙**: 스택은 Valid Parentheses, 힙은 Top K Frequent Elements($O(N \log K)$). C에서는 배열 기반 이진 힙을 직접 구현해야 합니다.
6. **트리 / 그래프 순회 (DFS/BFS)**: DFS는 재귀 또는 스택, BFS는 큐. 임베디드 관점에서는 **스택 크기가 작으므로 깊은 재귀 대신 명시적 스택**을 쓰는 이유를 말할 수 있으면 좋습니다.
7. **비트 조작**: Single Number(XOR), Number of 1 Bits, Reverse Bits(부록 A-3), Power of Two. 펌웨어 면접에서 특히 비중이 높습니다.

B. 'Pythonic'한 코드 활용법

핵심 로직은 직접 구현하되, 보조적인 데이터 저장·변환·탐색에 Python 내장 기능을 활용합니다. "정렬 구현하기"를

`list.sort()` 한 줄로 끝내면 평가할 것이 없어집니다.

1. dict 활용: Two Sum $O(n)$ 풀이

```
def twoSum(nums: list[int], target: int) -> list[int]:
    seen = {}                                # 값 -> 인덱스
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:                #  $O(1)$  평균 조회
            return [seen[complement], i]
        seen[num] = i
    return []
```

2. 리스트 컴프리헨션

```
# Non-Pythonic
new_list = []
for num in nums:
    if num > 3:
        new_list.append(num * num)

# Pythonic
new_list = [num * num for num in nums if num > 3]
```

3. 유용한 모듈

- `collections.defaultdict(int)` : 없는 키를 0으로 시작해 `d[key] += 1` 을 바로 사용.
- `collections.Counter(iterable)` : 빈도수를 O(n)에 계산. Valid Anagram에 유용.
- `collections.deque` : BFS 큐. `list.pop(0)` 은 O(n)이므로 `deque.popleft()` 를 사용.
- `heapq` : 최소 힙. 최대 힙이 필요하면 음수로 넣는 관용구를 사용.

C. 면접 진행 방식 (언어와 무관)

1. **요구사항 명확화**: 입력 범위, NULL/빈 입력, 오버플로 정책, 반환 형식을 먼저 묻습니다(부록 A의 모든 문제에 적용).
2. **예제로 손 풀이** 후 브루트포스 복잡도를 말하고 개선합니다.
3. **코드 작성** 중에는 생각을 소리 내어 설명합니다.
4. **테스트**: 정상 케이스, 경계값(0, 1, 최대/최소), 에러 입력을 직접 대입해 검증합니다.
5. **복잡도와 트레이드오프**(시간 vs 메모리, 코드 크기)를 정리합니다.

XI. 결론: "왜"를 이해하는 엔지니어

본 노트의 주제들은 임베디드 시스템 엔지니어 면접의 단골 질문이지만, 암기 테스트가 아닙니다. 면접관이 확인하려는 것은 다음과 같습니다.

- **volatile**: 컴파일러 최적화와 하드웨어의 충돌을 이해하는가.
- **static**: 링크 타임의 연결성, 저장 기간, 재진입성을 이해하는가.
- **const와 포인터**: "누가 무엇을 바꿀 수 있는가"를 타입으로 표현하고, 레지스터·API·메모리 배치에 적용할 수 있는가.
- **비트 조작과 MMIO**: read-modify-write의 원자성, W1C 레지스터, UB 없는 시프트를 알고 있는가.
- **메모리 레이아웃과 정수 규칙**: 변수가 어디에 놓이고, 정수 승격이 어떤 버그를 만드는지 아는가.
- **인터럽트와 동시성**: 레이스 조건을 시나리오로 설명하고 크리티컬 섹션·락프리 구조로 막을 수 있는가.
- **Mutex vs. Semaphore**: RTOS 스케줄링과 우선순위 역전을 이해하는가.
- **자료구조**: 동적 할당의 위험성과 정적 풀·링버퍼 대안을 설계할 수 있는가.
- **UB와 C++ 생명주기**: 표준이 보장하지 않는 동작과 RAII를 이해하는가.
- **통신 프로토콜**: 파형 수준에서 버스를 디버깅할 수 있는가.
- **알고리즘**: 복잡도와 경계 조건을 C로 정확하게 구현할 수 있는가.

결국 면접관이 찾는 인재는 "무엇(What)"을 아는 엔지니어가 아니라, "왜(Why)" 그 기술을 쓰고 "어떻게(How)" 동작하는지 근본 원리를 설명할 수 있는 엔지니어입니다. 부록의 기출 문제는 이 원리들이 실제 면접에서 어떤 모습으로 나오는지 보여줍니다.

부록 A. 기출 문제 풀이

실제 면접에서 받은 문제를 기억에 의존해 재구성하고, 당시 작성했던 코드(don-c-prac-master/interview_repro)를 리뷰한 뒤 검증된 모범 답안을 정리했습니다. 모든 모범 답안은 `cc -std=c11 -Wall -Wextra -Wconversion -Wsign-conversion -fsanitize=address,undefined` 로 경고 없이 컴파일되고 아래 테스트를 전부 통과했습니다.

#	문제	출처 (기록 기준)	핵심 주제
A-1	32비트 타이머 2개로 64비트 시간 읽기	Apple Core Bring-up 팀 2차 인터뷰 (2025-02-06)	MMIO, volatile, 레이스 컨디션, 원자적 읽기
A-2	atoi 구현	Anduril	문자열 파싱, 오버플로 검사, 경계값
A-3	Reverse Bits	Anduril	비트 조작, 룩업 테이블, 성능
A-4	Divide Two Integers	기록 없음 (LeetCode 29 유형)	시프트 나눗셈, INT_MIN, 오버플로
A-5	Palindrome (문자열/정수)	기록 없음 (LeetCode 9/125 유형)	투 포인터, 정수 오버플로

A-1. 32비트 타이머 2개로 64비트 시간 읽기

문제

하드웨어에 32비트 카운터 레지스터 두 개가 있다. `TIMER_LO` 는 계속 증가하는 free-running 카운터이고, `TIMER_LO` 가 `0xFFFFFFFF`에서 0으로 넘어갈 때마다 `TIMER_HI` 가 1 증가한다. 두 레지스터를 읽어 **일관된 64비트 시간 값**을 반환하는 함수를 작성하라.

(당시 기록에 문제 원문이 남아 있지 않아, 작성했던 코드와 메모를 바탕으로 재구성했습니다.)

면접관의 의도

- 두 레지스터를 따로 읽는 것 자체가 원자적이지 않다는 것을 알아차리는가.
- 읽는 도중 carry가 발생하는 구체적인 실패 시나리오를 말할 수 있는가.
- volatile MMIO 접근, 인터럽트를 끄지 않는 해법, 하드웨어 기능(latch)까지 생각하는가.

실패 시나리오: 단순히 두 번 읽으면

실제 카운트가 `0x0000_0001_FFFF_FFFF` 인 순간에 HI를 먼저 읽는 경우:

1. `hi = TIMER_HI` → `0x00000001`
2. (그 사이 카운터가 1 증가: LO는 `0x00000000`으로 wrap, HI는 `0x00000002`)
3. `lo = TIMER_LO` → `0x00000000`
4. 결과 `0x0000_0001_0000_0000` → 약 2^{32} 틱(수십 초~수 시간) 과거로 점프

순서를 바꿔 LO를 먼저 읽으면 `0x0000_0002_FFFF_FFFF` 가 되어 2^{32} 틱 미래로 점프합니다. 순서만 바뀌어서는 해결되지 않습니다.

원본 코드 리뷰

당시 두 가지 버전(Gemini 도움을 받은 답안)을 기록해 두었는데, 둘 다 문제가 있습니다.

- 버전 1 (32bit_two_timers_return_64bit_info.c): `TimerB.some_status_register`, `OVERFLOW_FLAG` 같은 가상의 이름을 써서 컴파일되지 않습니다. 또한 HI 레지스터를 읽는 대신 overflow 플래그를 폴링해 카운터를 static으로 누적하는데, 함수가 overflow 주기보다 드물게 호출되면 overflow를 놓치고, ISR이 같은 플래그를 처리하면 경합이 생깁니다. `printf("%lu", uint64_t)` 도 이식성이 없어 `PRIu64` 를 써야 합니다.
- 버전 2 (32bit_two_timers_return_64bit_info2.md): LO를 두 번 읽어 `a1 != a2` 이면 "overflow가 났다"고 판단합니다. 그러나 동작 중인 카운터는 읽을 때마다 값이 달라지므로 이 조건은 거의 항상 참이며 overflow 여부와 무관합니다. 결과도 여전히 일관성이 보장되지 않습니다. 카운터가 읽기마다 증가하는 하드웨어 시뮬레이션에서 wrap 경계 근처 60가지 경우를 돌려 보니 버전 2는 6번 틀린 값을, 아래 모범 답안은 0번 틀린 값을 반환했습니다.

모범 답안: HI를 앞뒤로 읽고 바뀌었으면 재시도

```
#include <stdint.h>

#define TIMER_BASE 0x40010000UL
#define TIMER_LO (*(const volatile uint32_t *) (TIMER_BASE + 0x00)) /* 읽기 전용 */
#define TIMER_HI (*(const volatile uint32_t *) (TIMER_BASE + 0x04))

uint64_t read_timer64(void) {
    uint32_t hi, lo;
    do {
        hi = TIMER_HI;
        lo = TIMER_LO;
    } while (hi != TIMER_HI); /* LO를 읽는 동안 carry가 발생했다면 다시 */
    return ((uint64_t)hi << 32) | lo;
}
```

왜 올바른가: HI를 읽은 뒤 LO를 읽고 다시 HI를 읽었을 때 값이 같다면, 그 구간에서 LO의 wrap(= HI 증가)이 일어나지 않았다는 뜻이므로 lo는 hi와 같은 "시대"의 값입니다. HI가 바뀌었다면 carry가 끼어든 것이므로 다시 읽습니다. carry는 2^{32} 틱마다 한 번이므로 재시도는 사실상 최대 1회이고 루프는 반드시 끝납니다.

설명하며 짚을 포인트:

- `(uint64_t)hi << 32` 에서 캐스트가 먼저여야 합니다. `hi << 32` 는 32비트 값을 32만큼 시프트하는 UB입니다.
- 레지스터 매크로에 `const volatile` 을 써서 읽기 전용임을 타입으로 표현했습니다(II-D).
- 인터럽트를 끄지 않으므로 지연(latency)에 영향이 없고, static 상태가 없어 ISR과 태스크에서 동시에 호출해도 안전합니다 (재진입 가능).

꼬리 질문과 답

- "하드웨어가 LO wrap과 HI 증가를 같은 클럭 에지에서 하지 않는다면?" carry 전파 지연으로 LO=0인데 HI가 아직 이전 값인 짧은 창이 있을 수 있습니다. 가장 좋은 답은 데이터시트에서 latch/snapshot 기능을 확인하는 것입니다. 많은 타이머 IP는

LO를 읽는 순간 HI를 shadow 레지스터에 래치해 두므로 "LO 먼저, 그다음 HI" 순서로 읽으면 원자적 스냅샷이 됩니다. 이런 기능이 있는지 먼저 묻는 것 자체가 좋은 인상을 줍니다.

- "HI 레지스터가 없고, 32비트 타이머 하나와 overflow 인터럽트만 있다면?" 소프트웨어로 상위 32비트를 확장합니다. 이때는 overflow가 발생했지만 ISR이 아직 실행되지 못한(pending) 순간을 처리해야 합니다.

```
static volatile uint32_t g_overflows;          /* overflow ISR에서만 증가 */

void TIMER_OVF_IRQHandler(void) {
    TIMER_CLEAR_OVF_FLAG();
    g_overflows++;
}

uint64_t read_timer64_sw(void) {
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    uint32_t ovf = g_overflows;
    uint32_t cnt = TIMER_CNT;
    if (TIMER_OVF_PENDING() && cnt < 0x80000000UL) {
        ovf++; /* wrap은 됐는데 ISR이 아직 못 돈 경우: 카운트가 작으면 wrap 이후 값 */
    }
    __set_PRIMASK(primask);
    return ((uint64_t)ovf << 32) | cnt;
}
```

`cnt < 0x80000000` 조건은 "pending 플래그가 cnt를 읽기 전에 켜졌는지(cnt는 wrap 후라 작음), 후에 켜졌는지(cnt는 wrap 전이라 큼)"를 구분합니다.

- "64비트 틱으로 경과 시간을 계산하려면?" `uint64_t elapsed = now - start;` unsigned 뽕셈이므로 wrap에도 안전합니다(IV-C). 32비트 틱을 쓴다면 최대 측정 구간이 2^{32} 틱으로 제한된다는 점도 말합니다.
- "이 함수를 1kHz로 호출하는데 느리다면?" 레지스터 읽기 3번은 매우 싸지만, 더 줄이려면 하드웨어 latch 사용, 또는 SysTick 기반 64비트 소프트웨어 틱(V-B)으로 대체합니다.

A-2. atoi 구현

문제

문자열을 정수로 변환하는 `int my_atoi(const char *str)` 을 구현하라. 앞쪽 공백을 건너뛰고, 선택적인 부호(+/-)를 처리하며, 숫자가 아닌 문자를 만나면 멈춘다. int 범위를 넘으면 INT_MAX 또는 INT_MIN으로 포화(saturate)한다. (LeetCode 8 String to Integer와 같은 규칙)

면접관의 의도

- 요구사항을 먼저 묻는가: NULL 입력, 빈 문자열, "+-2", 오버플로 정책(포화 vs 에러 보고), 뒤에 붙은 문자 처리.

- **오버플로를 발생시키기 전에** 검사하는가. `result * 10 + d` 를 계산한 뒤 검사하면 이미 signed 오버플로 UB입니다.
- `INT_MIN(-2147483648)` 처럼 절대값이 `INT_MAX`보다 큰 경계를 올바르게 처리하는가.

원본 코드 리뷰 (`anduril_atoi.c`)

알고리즘 자체는 올바릅니다. 공백·부호·숫자 처리와 "곱하기 전에 검사하는" 오버플로 체크(`result > INT_MAX / 10 || (result == INT_MAX / 10 && digit > INT_MAX % 10)`)가 잘 되어 있고, `-2147483648` 은 `digit 8 > 7` 조건에서 `INT_MIN`을 반환해 우연이 아니라 정확하게 처리됩니다. 개선할 점:

- **테스트 코드가 컴파일되지 않습니다:** `char str11[] = NULL;` 은 배열을 `NULL`로 초기화하는 문법 오류입니다(`const char *str11 = NULL;` 이어야 함). 또 `printf("%s", NULL)` 은 UB이므로 `NULL` 테스트는 출력 없이 반환값만 확인해야 합니다.
- `isspace(str[i])`, `isdigit(str[i])` 에 음수 `char`를 넘기면 UB: `char`가 signed인 플랫폼에서 `0x80` 이상 바이트 (UTF-8 한글 등)는 음수가 되어 `ctype` 함수의 입력 범위를 벗어납니다. `(unsigned char)` 로 캐스트해야 합니다.
- 인덱스 `int i` 대신 포인터를 전진시키면 더 간결하고, `size_t` 대 `int` 변환 경고도 사라집니다.

모범 답안

```
#include <ctype.h>
#include <limits.h>
#include <stddef.h>

int my_atoi(const char *s) {
    if (s == NULL) {
        return 0;
    }
    while (isspace((unsigned char)*s)) {
        s++;
    }
    int sign = 1;
    if (*s == '+' || *s == '-') {
        if (*s == '-') {
            sign = -1;
        }
        s++;
    }
    int result = 0;
    while (isdigit((unsigned char)*s)) {
        int d = *s - '0';
        if (result > (INT_MAX - d) / 10) { /* result * 10 + d > INT_MAX 를 미리 검사 */
            return (sign == 1) ? INT_MAX : INT_MIN;
        }
        result = result * 10 + d;
        s++;
    }
    return sign * result;
}
```

`result > (INT_MAX - d) / 10` 은 원본의 두 줄짜리 조건과 동치인 한 줄 검사입니다. `-2147483648` 은 누적값이 `INT_MAX`를 넘는 시점에 `INT_MIN`으로 포화되므로 따로 처리할 필요가 없습니다.

테스트 케이스 (모두 통과)

입력	기대값	확인하는 것
"42"	42	기본
" -42"	-42	공백 + 음수
"4193 with words"	4193	뒤 문자에서 정지
"words and 987"	0	숫자로 시작하지 않음

입력	기대값	확인하는 것
" <code>+0123</code> "	123	+ 부호, 앞자리 0
" <code>+ -2</code> "	0	부호 두 개
" <code>2147483647</code> " / " <code>2147483648</code> "	INT_MAX / INT_MAX	경계, 오버플로
" <code>-2147483648</code> " / " <code>-91283472332</code> "	INT_MIN / INT_MIN	음수 경계, 오버플로
"", " ", NULL	0	빈 입력
" <code>\xE9</code> " " <code>12</code> "	0	0x80 이상 바이트 (ctype UB 방지)

꼬리 질문과 답

- "0을 반환했을 때 '0'이라는 입력인지 에러인지 어떻게 구분하나?" 표준 `atoi` 의 약점입니다. `strtol` 처럼 끝 위치와 에러를 함께 반환하는 API로 설계합니다: `bool parse_int32(const char *s, int32_t *out, const char **end);` 펌웨어의 명령어 파서(예: UART 콘솔 `set speed 1200`)에서는 이 형태가 필수입니다.
- "16진수(`0x1F`)도 받으려면?" base 인자를 추가하고 `'a'-'f'` 를 10~15로 매핑합니다. 오버플로 검사의 10을 base로 일반화합니다.
- "ctype.h 없이?" `(unsigned)(c - '0') <= 9u` 는 분기 하나로 숫자를 판별하는 관용구입니다.
- 복잡도: 시간 $O(n)$, 공간 $O(1)$.

A-3. Reverse Bits

문제

32비트 부호 없는 정수의 비트 순서를 뒤집어라. 예: 43261596 (0b0000000101001010000001111010011100) → 964176192 (0b0011100101111000000101001010000000). (LeetCode 190)

면접관의 의도

- 기본 시프트 루프를 정확히 작성하는가 (unsigned 사용, 32번 반복).
- "이 함수가 초당 수백만 번 호출된다면?"이라는 성능 후속 질문에 록업 테이블이나 분할 정복 방식으로 답할 수 있는가.
- 임베디드 실무와 연결할 수 있는가: SPI LSB-first 디바이스, CRC의 reflected 입력/출력, 비트 순서가 반대인 하드웨어 버스.

원본 코드 리뷰 (`anduril_reverseBit.c`)

두 구현 모두 **올바르게 동작합니다**(실행 결과 43261596 → 964176192, 5 → 2684354560 = 0xA0000000 확인). 사소한 개선점은 `printf("%u", uint32_t)` 대신 `<inttypes.h>` 의 `PRId32` 를 쓰는 것(`uint32_t`가 unsigned long인 플랫폼 대비)입니다. 최적화 버전은 16/8/4/2/1비트 순서로 스왑하는데, 아래처럼 1/2/4/8/16 순서로 해도 결과가 같습니다(각 단계가 서로 다른 비트 쌍을 교환하는 독립적인 치환이기 때문).

모범 답안 1: 기본 루프 (O(32))

```
#include <stdint.h>

uint32_t reverse_bits(uint32_t n) {
    uint32_t r = 0;
    for (int i = 0; i < 32; i++) {
        r = (r << 1) | (n & 1u);    /* n의 LSB를 r의 LSB로 밀어 넣음 */
        n >>= 1;
    }
    return r;
}
```

모범 답안 2: 분할 정복 스왑 (분기 없음, 연산 5단계)

```
uint32_t reverse_bits_swap(uint32_t n) {
    n = ((n >> 1) & 0x55555555u) | ((n & 0x55555555u) << 1);    /* 인접한 1비트끼리 교환 */
    n = ((n >> 2) & 0x33333333u) | ((n & 0x33333333u) << 2);    /* 2비트 묶음끼리 교환 */
    n = ((n >> 4) & 0x0F0F0F0Fu) | ((n & 0x0F0F0F0Fu) << 4);    /* 니블끼리 교환 */
    n = ((n >> 8) & 0x00FF00FFu) | ((n & 0x00FF00FFu) << 8);    /* 바이트끼리 교환 */
    n = (n >> 16) | (n << 16);    /* 워드 반쪽 교환 */
    return n;
}
```

모범 답안 3: 니블 룩업 테이블 (16바이트 테이블, 8번 반복)

```
static const uint8_t nib_rev[16] = {
    0x0, 0x8, 0x4, 0xC, 0x2, 0xA, 0x6, 0xE,
    0x1, 0x9, 0x5, 0xD, 0x3, 0xB, 0x7, 0xF
};

uint32_t reverse_bits_lut(uint32_t n) {
    uint32_t r = 0;
    for (int i = 0; i < 8; i++) {
        r = (r << 4) | nib_rev[n & 0xFu];    /* 하위 니블을 뒤집어 상위부터 채움 */
        n >>= 4;
    }
    return r;
}
```

세 구현은 0, 1, 0x80000000, 0xFFFFFFFF, 0x12345678 등 경계값에서 서로 같은 결과를 내고, 두 번 뒤집으면 원래 값이 되는 것(`reverse(reverse(x)) == x`)까지 테스트로 확인했습니다.

꼬리 질문과 답

- "가장 빠른 방법은?" 호출 빈도와 메모리에 따라 다릅니다. 8비트 테이블(256바이트, `static const` 로 플래시)은 4번 조회로 끝나고, 분할 정복은 테이블 없이 분기도 없습니다. ARMv6T2 이상(Cortex-M3/M4 등)에는 `RBIT` 명령이 있어 한 명령으로 끝나며, Clang은 `__builtin_bitreverse32` 를 제공합니다. "측정해 보고 결정하겠다"는 태도도 좋은 답입니다.
- "8비트만 뒤집는다면?" `((b * 0x0802LU & 0x22110LU) | (b * 0x8020LU & 0x88440LU)) * 0x10101LU >> 16` 같은 트릭도 있지만, 면접에서는 니블 테이블 두 번 조회가 가장 읽기 쉽고 확실합니다.
- "CRC에서 왜 비트 반전이 나오나?" UART처럼 LSB부터 전송되는 링크에서 쓰는 CRC(예: CRC-32 Ethernet)는 "reflected" 알고리즘을 사용하며, 입력 바이트와 최종 CRC를 비트 반전하는 것과 동등합니다.
- 복잡도: 루프 $O(32) = O(1)$, 테이블 $O(8) = O(1)$, 공간 $O(1)$.

A-4. Divide Two Integers

문제

곱셈(`*`), 나눗셈(`/`), 나머지(`%`) 연산자 없이 두 정수의 몫을 구하라. 결과는 0 방향으로 버림(truncate)하며, 32비트 signed 범위를 넘으면 `INT_MAX`를 반환한다. (LeetCode 29)

면접관의 의도

- 뺄셈 반복($O(\text{몫})$)에서 시프트를 이용한 $O(\log n)$ 긴 나눗셈으로 개선할 수 있는가.
- `INT_MIN` 처리: `-INT_MIN` 은 `int`로 표현할 수 없고, `INT_MIN / -1` 은 유일한 오버플로 케이스입니다.
- 부호 처리와 시프트 시 signed 오버플로(UB)를 피하는가.
- 임베디드 연결: 하드웨어 나눗셈기가 없는 MCU(Cortex-M0/M0+)에서 나눗셈이 어떻게 구현되는지.

원본 코드 리뷰 (`divide_two_integer.c`)

뺄셈 반복 → 두 배씩 불리기 → 시프트로 개선하는 흐름은 면접에서 설명하기 좋은 순서입니다. 하지만 실제 컴파일·실행해 보니 다음 문제가 있었습니다.

- 컴파일 에러: `#include <stdio.h>` 가 없어 C99 이후 컴파일러에서 `printf` 암시적 선언 에러가 납니다.
- 주석의 기대값 오류: `divider(1000, 3)` 옆에 `// return 3;` 이라고 적혀 있지만 실제 몫은 333입니다.
- 음수 미지원: `divider(-7, 3)` 은 루프에 들어가지 않고 0을 반환합니다(정답 -2).
- divisor가 0 이하면 무한 루프: `divider(5, 0)` 은 `a >= 0` 이 계속 참이라 끝나지 않습니다.
- signed 오버플로 UB와 무한 루프: `divider_perfect_2(INT_MAX, 1)` 에서 `temp << 1` 이 1073741824를 넘어 `int` 오버플로(UBSan 검출)가 발생하고, 음수가 된 `temp` 때문에 `(temp << 1) <= dividend` 가 계속 참이 되어 실제로 무한 루프에 빠졌습니다. `temp + temp` 버전도 같은 문제가 있습니다.

모범 답안: unsigned 긴 나눗셈

```
#include <limits.h>
#include <stdbool.h>
#include <stdint.h>

int divide(int dividend, int divisor) {
    if (divisor == 0) {
        return (dividend >= 0) ? INT_MAX : INT_MIN;    /* 정책은 면접관과 합의 */
    }
    if (dividend == INT_MIN && divisor == -1) {
        return INT_MAX;                                /* 유일한 오버플로 케이스 */
    }
    bool negative = (dividend < 0) != (divisor < 0);

    /* 절대값을 unsigned로: |INT_MIN| = 2^31 도 표현 가능 */
    uint32_t a = (dividend < 0) ? 0u - (uint32_t)dividend : (uint32_t)dividend;
    uint32_t b = (divisor < 0) ? 0u - (uint32_t)divisor : (uint32_t)divisor;

    uint32_t q = 0;
    for (int shift = 31; shift >= 0; shift--) {
        if ((a >> shift) >= b) {    /* a >= (b << shift) 를 오버플로 없이 비교 */
            a -= b << shift;
            q |= 1u << shift;
        }
    }
    return negative ? (int)(0u - q) : (int)q;
}
```

핵심 아이디어: 손으로 하는 긴 나눗셈과 같습니다. 가장 큰 자리(2^{31})부터 "b를 이만큼 시프트한 값을 a에서 뺄 수 있는가?"를 확인하고, 뺄 수 있으면 몫의 그 비트를 1로 세웁니다. `b << shift`를 직접 계산해 비교하면 오버플로가 나므로, `a >> shift`와 b를 비교하는 것이 요령입니다. 루프는 항상 32번이므로 시간 복잡도는 $O(32) = O(1)$ 입니다.

unsigned를 쓰는 이유: `-INT_MIN`은 int로 표현할 수 없어 UB이지만, `0u - (uint32_t)INT_MIN`은 모듈로 연산으로 정확히 2147483648이 됩니다. 결과를 int로 되돌리는 `(int)(0u - q)` 변환은 q가 2^{31} 일 때(`INT_MIN / 1`) 구현 정의 동작이지만, 모든 2의 보수 플랫폼에서 `INT_MIN`이 되며 C23부터는 2의 보수가 표준입니다.

테스트 (모두 통과)

(10, 3)=3, (7, -3)=-2, (-7, 3)=-2, (-7, -3)=2, (0, 1)=0, (1000, 3)=333, (INT_MAX, 1), (INT_MAX, -1), (INT_MIN, 1), (INT_MIN, 2), (INT_MIN, INT_MIN)=1, (INT_MAX, INT_MIN)=0, (INT_MIN, INT_MAX)=-1, (INT_MIN, -1)=INT_MAX, 그리고 -3000~3000 × -40~40의 모든 조합을 C의 `/` 연산 결과와 비교해 일치함을 확인했습니다.

꼬리 질문과 답

- "Cortex-M0에는 나눗셈 명령이 없는데 `a / b` 는 어떻게 실행되나?" 컴파일러가 런타임 라이브러리 함수 (`__aeabi_idiv`, `__aeabi_uidiv`)를 호출하며, 내부는 위와 같은 시프트-뺄셈 방식입니다. 따라서 ISR이나 제어 루프에서 나눗셈은 비쌉니다.
- "상수로 나누면?" 컴파일러는 `x / 10` 같은 상수 나눗셈을 **역수 곱셈 + 시프트**(magic number)로 바꿉니다. 그래서 나눗셈 명령이 없는 MCU에서도 상수 나눗셈은 빠릅니다.
- "2의 거듭제곱으로 나눌 때 `x >> 1` 과 `x / 2` 는 같은가?" 음수에서는 다릅니다. `-7 / 2` 는 0 방향 버림으로 -3이지만, `-7 >> 1` 은 (대부분의 산술 시프트 구현에서) -4이고, 음수의 오른쪽 시프트는 구현 정의 동작입니다. 고정소수점 연산에서 자주 나오는 버그입니다.
- "나머지도 필요하면?" 루프가 끝난 뒤의 `a` 가 나머지의 절대값이며, 부호는 피제수(dividend)를 따릅니다.

A-5. Palindrome (문자열과 정수)

문제

(1) 문자열이 앞뒤로 같은지 판별하라. (2) 정수가 앞뒤로 같은지 판별하라. 음수는 palindrome이 아니다. (LeetCode 9 Palindrome Number, 확장으로 LeetCode 125 Valid Palindrome)

면접관의 의도

- 투 포인터로 $O(n)$ 시간, $O(1)$ 공간 풀이를 작성하는가.
- 정수 버전에서 **숫자를 뒤집다가 오버플로**가 날 수 있음을 알아차리는가.
- 문자열로 변환하지 않고(추가 메모리 없이) 풀 수 있는가.

원본 코드 리뷰 (`palindrome.c`)

- 문자열 버전은 **올바릅니다**. NULL 처리, 빈 문자열, 투 포인터가 잘 되어 있습니다. `int len = strlen(str)` 은 `size_t` 를 `int`로 줄이는 변환이므로 `size_t` 를 쓰는 편이 좋습니다(`-Wconversion` 경고).
- 정수 버전에는 **오버플로 버그가 있습니다**. 숫자 전체를 뒤집으므로 `2147483647` 을 넣으면 뒤집은 값 `7463847412`가 `INT_MAX`를 넘어 **signed 오버플로 UB**가 됩니다. palindrome인 큰 수(예: `2147447412`)는 뒤집어도 원래 값이라 문제없지만, palindrome이 아닌 큰 수에서 UB가 발생합니다.
- `#define false (0)` 대신 C99 `<stdbool.h>` 를 사용하는 것이 표준적입니다.

모범 답안 1: 문자열 (포인터 투 포인터)

```
#include <stdbool.h>
#include <string.h>

bool is_str_palindrome(const char *s) {
    if (s == NULL) {
        return false;
    }
    size_t n = strlen(s);
    if (n == 0) {
        return true;
    }
    const char *l = s;
    const char *r = s + n - 1;
    while (l < r) {
        if (*l++ != *r--) {
            return false;
        }
    }
    return true;
}
```

모범 답안 2: 정수 (절반만 뒤집기, 오버플로 없음)

```
bool is_int_palindrome(int x) {
    /* 음수, 그리고 0이 아니면서 끝자리가 0인 수(예: 10)는 palindrome 불가 */
    if (x < 0 || (x % 10 == 0 && x != 0)) {
        return false;
    }
    int rev = 0;
    while (x > rev) {
        /* 뒤쪽 절반만 뒤집음 → rev는 원래 수의 절반 자릿수 */
        rev = rev * 10 + x % 10;
        x /= 10;
    }
    return x == rev || x == rev / 10; /* 자릿수가 짝수 || 홀수(가운데 자리 제거) */
}
```

왜 오버플로가 없는가: rev는 원래 수의 자릿수 절반까지만 커지므로 INT_MAX(10자리)의 절반인 5자리를 넘지 않습니다. 예를 들어 12321은 x=12, rev=123에서 멈추고 `x == rev / 10` (12 == 12)으로 판정합니다. 끝자리가 0인 수를 먼저 거르는 이유는, 10처럼 `x=1, rev=0`에서 잘못 참이 나오는 경우를 막기 위해서입니다.

모범 답안 3: 영숫자만 비교, 대소문자 무시 (LeetCode 125)

```
#include <ctype.h>

bool is_alnum_palindrome(const char *s) {
    if (s == NULL) {
        return false;
    }
    const char *l = s;
    const char *r = s + strlen(s);          /* r은 비교할 문자 다음 위치 */
    while (l < r) {
        if (!isalnum((unsigned char)*l)) { l++; continue; }
        if (!isalnum((unsigned char)r[-1])) { r--; continue; }
        if (tolower((unsigned char)*l) != tolower((unsigned char)r[-1])) {
            return false;
        }
        l++;
        r--;
    }
    return true;
}
```

테스트 (모두 통과)

- 문자열: "racecar" 참, "hello" 거짓, "" 참, "a" 참, "abba" 참, NULL 거짓
- 정수: 121 참, 123 거짓, 12321 참, -121 거짓, 5 참, 0 참, 10 거짓, 1221 참, INT_MAX 거짓(UB 없음), 2147447412 참
- 영숫자: "A man, a plan, a canal: Panama" 참, "race a car" 거짓, " " 참, ".", " 참, "0P" 거짓

꼬리 질문과 답

- "연결 리스트가 palindrome인지 O(1) 공간으로 판별하라." (LeetCode 234) Slow/Fast 포인터로 중간을 찾고, 뒤쪽 절반을 제자리에서 뒤집은 뒤 앞쪽과 비교하고, 원래대로 다시 뒤집어 복구합니다. 입력을 망가뜨리지 않는 것까지 말하면 좋습니다.
- "문자열이 UTF-8이라면?" 바이트 단위 비교는 멀티바이트 문자를 깨뜨립니다. 코드 포인트 단위로 디코딩해야 하며, 조합 문자까지 고려하면 정규화가 필요합니다.
- "문자 하나를 지워서 palindrome을 만들 수 있나?" (LeetCode 680) 처음 불일치 지점에서 왼쪽을 건너뛴 경우와 오른쪽을 건너뛴 경우를 각각 한 번씩 검사합니다. O(n).
- 복잡도: 문자열 O(n) 시간 O(1) 공간, 정수 O(log10 x) 시간 O(1) 공간.

부록 B. draft-1 정오표

draft-1에서 기술적으로 틀렸거나 오해의 소지가 있던 부분과 draft-2의 수정 내용입니다.

#	draft-1 위치	draft-1 내용	문제	draft-2 수정
1	I-A-2 (원자성)	"8비트 MCU에서 volatile uint32_t 읽기는 4개의 8비트 읽기(LDRB)"	LDRB는 32비트 ARM 명령이라 8비트 MCU 예시와 맞지 않음. std::atomic은 C++ 전용	AVR/Cortex-M 예시로 교체, C 해법(크리티컬 섹션, stdatomic) 추가
2	I-A-2 (메모리 배리어)	<code>asm volatile ("" : : : "memory");</code> 같은 배리어 사용	존재하지 않는 문법(생성 AI 답변의 잔재). 순서 문제의 실제 원인 설명 부족	관찰 가능한 동작 개념으로 재설명, 배리어 문법 삭제
3	I-B-2 (static 지역 변수)	초기화는 "첫 호출 시 또는 프로그램 시작 시"	C와 C++의 규칙이 다른데 섞여 있음	C(main 전 상수 초기화)와 C++(첫 통과 시 동적 초기화) 구분, 재진입 불가 함정 추가
4	II-A-3 (매크로 효율)	매크로를 쓰면 <code>STR R0, [0x40013804]</code> 처럼 주소가 명령에 직접 포함	ARM은 32비트 절대 주소를 명령어에 담을 수 없어 리터럴 풀 로드가 필요. const 포인터 상수도 같은 코드 생성	II-D에서 const 포인터와 비교하여 정확히 설명
5	II-B-2 (매크로 괄호)	괄호가 없으면 <code>SET_BIT(my_reg, pin_num + 1)</code> 이 잘못 계산됨	<code>+</code> 가 <code><<</code> 보다 우선순위가 높아 실제로는 올바르게 계산됨(직접 확인)	<code>x & 0x7</code> 반례로 교체, <code>1 << 31</code> UB와 1ULL 설명 추가
6	II-B-3 (비트 카운팅)	$O(32)$ "(또는 $O(\log N)$)"	고정 폭 정수에서는 $O(32)=O(1)$ 로 표현하는 것이 정확	$O(k)$ vs $O(32)$ 비교 유지, popcount 내장 함수 추가
7	II-B-4 (엔디안 스왑)	내장 함수가 수동 구현보다 "압도적으로 빠르다"	최신 GCC/Clang은 수동 패턴도 REV 한 명령으로 최적화	"의도가 명확하고 최적화가 보장된다"로 수정
8	IV-A-2 (정적 풀)	<code>static struct Node g_node_pool;</code>	배열 크기 <code>[MAX_NODES]</code> 누락 (원문 누락)	<code>g_node_pool[MAX_NODES]</code> 와 초기화 함수까지 완성
9	V-A-2 (printf UB)	"printf는 스택의 마지막부터 값을 가져오므로 출력은..." / 시퀀스 포인트 "함수 호출이 완료된 직후"	인자는 레지스터로 전달될 수 있고, 시퀀스 포인트는 호출 완료 후가 아니라 호출 직전	평가 순서와 전달 방식 분리, 시퀀스 포인트 목록 수정, C++17 차이 추가
10	VI-B-1 (Two Sum 코드)	<code>return</code> (빈 반환), <code>new_list =</code>	<code>[]</code> 누락으로 문법 오류 (원문 누락)	<code>return []</code> , <code>new_list = []</code> 로 복원